

JOI 2014-2015 本選1

鉄道旅行(Railroad Trip) 解説

三谷 庸 (wo)

問題概要

- $1, 2, \dots, N$ という駅が順番に並んでいる。
- 駅 i と駅 $i+1$ の間には鉄道が走っている。
- $P_1, P_2, \dots, P_M$ の順に旅行する。
- 駅 i と駅 $i+1$ の間の移動には
- A_i 円必要
- あらかじめ C_i 円払っておくと B_i 円で移動可能
- 最小の費用を求める

小課題1 (20 点)

- 訪れる都市の数 = 2

小課題1 (20 点)

- 訪れる都市の数 = 2
- 各鉄道には 1 回しか乗らない
- 満点をとれる解法が思いつかなくても、単純な解法で部分点を取れることがあります。

小課題1 (20 点)

- 訪れる都市の数 = 2
- 各鉄道には 1 回しか乗らない
- 各鉄道に対して
- 紙の切符で乗る: A_i 円
- IC切符で乗る: $B_i + C_i$ 円
- の安い方をとる

小課題2 (30 点)

- 鉄道の数 $\leq 1\,000$
- 訪れる都市の数 $\leq 1\,000$

小課題 2 (30 点)

- 各鉄道についてICカードを使うか紙の切符を使うかは独立に決められる。
- ICカードを買った鉄道に乗るときは、毎回ICカードを使って乗る。

小課題 2 (30 点)

- 各鉄道について、その鉄道に乗る回数だけ考えればよい。
- 鉄道 i に n_i 回乗るとすると、その鉄道に払うべき金額は

$$\min(A_i * n_i , B_i * n_i + C_i) \text{ 円}$$

小課題 2 (30 点)

- 各鉄道を使う回数を数える。
- 鉄道に対応する整数配列を用意し、
- 都市 x から都市 y ($x < y$) に移動するときに
- 鉄道 $x, x+1, \dots, y-1$ を使う回数を 1 ずつ増やす。
- $x > y$ ならば x と y を入れ替えればよい。

小課題 2 (30 点)

- 計算量 (計算にかかる時間) は？
- 「都市 x から都市 y へ行く ($x < y$) 」という処理をするには 1 足す操作を $y-x$ ($< N$) 回行う必要がある。

小課題 2 (30 点)

- $O(N)$ の処理を M 回やる。
- 計算量は $O(NM)$
- $NM \leq 1\,000\,000$ なので、間に合いそうであると分かります。

満点解法

- 配列の x 番目から y 番目に 1 を足すという操作を高速にやりたい。

満点解法

- 配列の x 番目から y 番目に 1 を足すという操作を高速にやりたい。

累積和

累積和

	1	1	1	1	
--	----------	----------	----------	----------	--



	+1				-1
--	-----------	--	--	--	-----------

累積和

- $\text{count}[x], \text{count}[x+1], \dots, \text{count}[y]$ に $+1$ をしたいとする。
- ひとつずつ $+1$ していくかわりに、
- $\text{count}[x]$ に $+1$ し、 $\text{count}[y+1]$ に -1 する。
- その後・・・

累積和

	+1				-1
--	-----------	--	--	--	-----------



	1	1	1	1	
--	----------	----------	----------	----------	--

累積和

- 累積和をとる
- 各 i に対して、
- $\text{count}[0] + \text{count}[1] + \dots + \text{count}[i]$
- を求める

累積和

- 小さい i から順番に、
 - $\text{sum}[i] = \text{sum}[i-1] + \text{count}[i]$
 - のように処理していくと高速に求まる
-
- このような考え方を累積和といい、プログラミングコンテストで頻出です。

満点解法

- 問題の解き方に戻って、
- 方針は小課題 2 の解法と同じ
- 鉄道を使う回数を求める時に累積和を使う。

満点解法

- 計算量は？
- 始点に $+1$, 終点の一つ後に -1 する: $O(M)$
- 累積和を求める: $O(N)$
- 全部で $O(N+M)$ で解ける。
- $N, M \leq 100\,000$ でも間に合う。

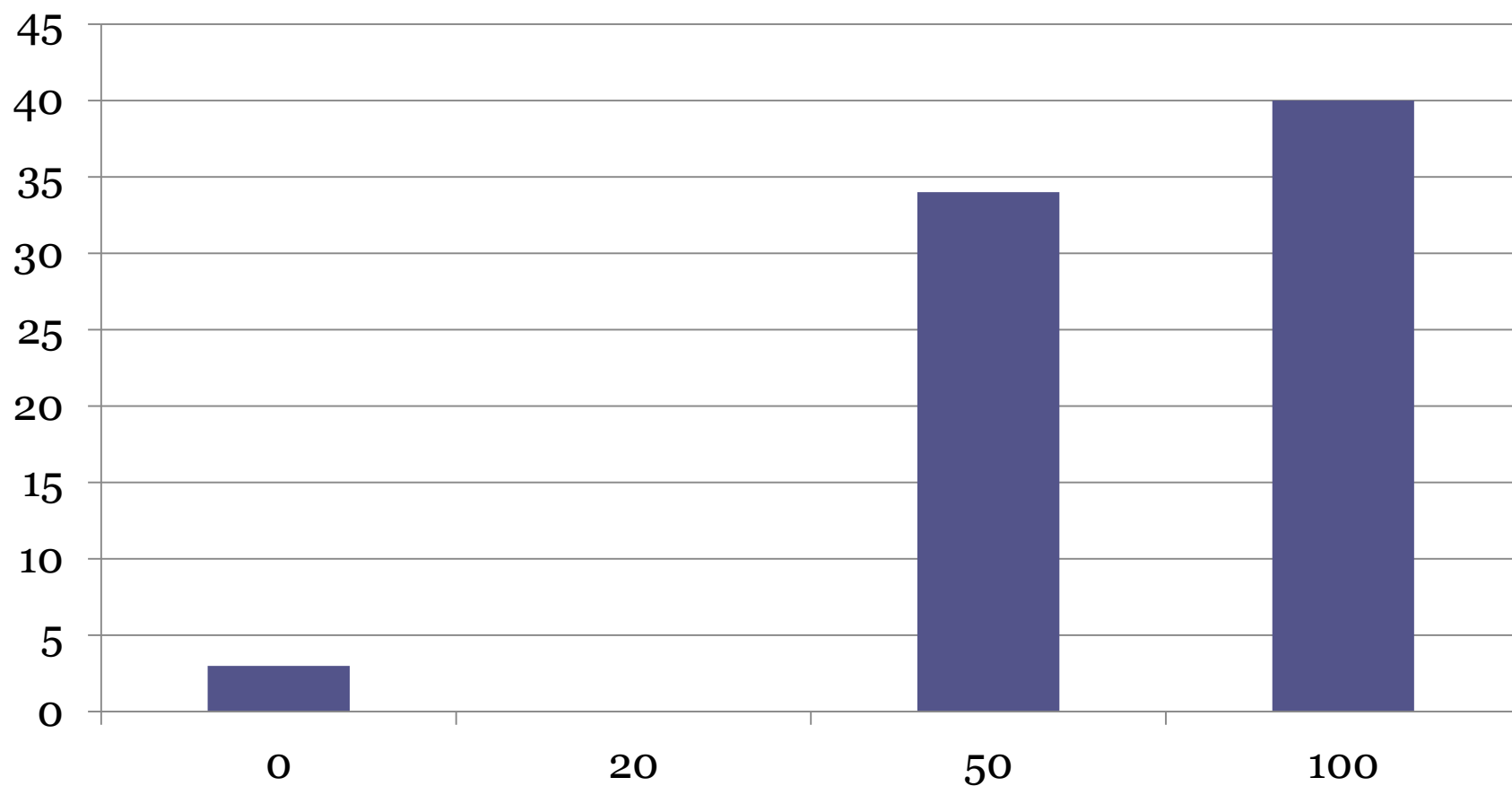
注意点

- 答えの値は大きくなります。
- 鉄道に乗るたびに毎回およそ 100 000 円とられることがあります。
- 99 999 日間毎日 99 999 本の鉄道に乗ることがあり得る。

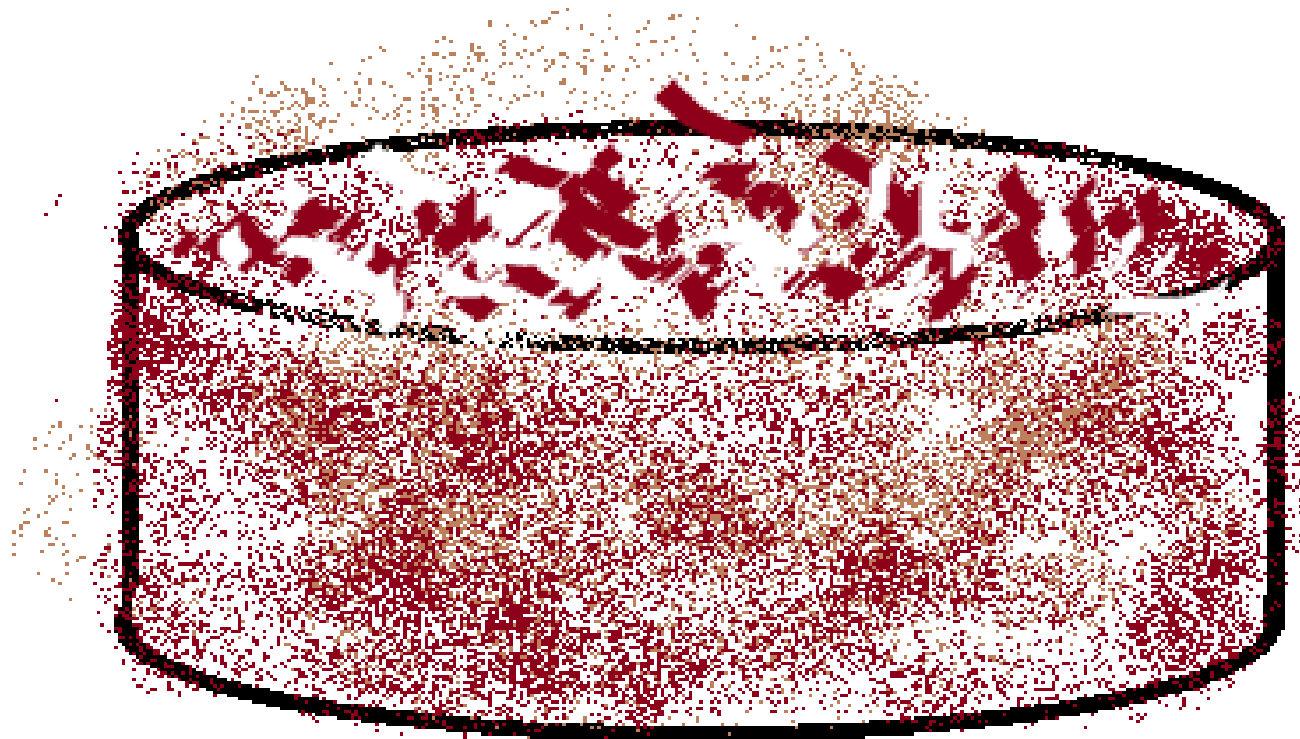
注意点

- その時の総金額はおよそ
- $100\ 000 * 100\ 000 * 100\ 000 = 10^{15}$ 円
- int に収まらない
- long long を使いましょう。
- 使い方は Technical info の通り

得点分布



Cake 2 解説



解説: 秀 郁未 (tozangezan)

"2" ...?

ケーキの切り分け (Cake)

JOI ちゃんと IOI ちゃんは双子の兄妹である。JOI くんは最近お菓子作りに凝っていて、今日も JOI くんはケーキを焼いて食べようとしたのだが、焼きあがったところで匂いをかぎつけた IOI ちゃんが来たので 2 人でケーキを分けることになった。

ケーキは円形である。ある点から放射状に切り目を入れ、ケーキを N 個のピースに切り分け、ピースに 1 から N まで反時計回りに番号をつけた。つまり、 $1 \leq i \leq N$ に対し、 i 番目のピースは $i-1$ 番目と $i+1$ 番目のピースと隣接している (ただし 0 番目は N 番目、 $N+1$ 番目は 1 番目とみなす)。 i 番目のピースの大きさは A_i だったが、切り方がとても下手だったので A_i はすべて異なる値になった。

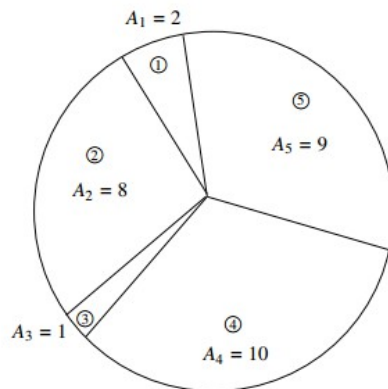


図 1: ケーキの例 ($N = 5, A_1 = 2, A_2 = 8, A_3 = 1, A_4 = 10, A_5 = 9$)

2

ケーキの切り分け 2 (Cake 2)

JOI ちゃんと IOI ちゃんは双子の兄妹である。JOI くんは最近お菓子作りに凝っていて、今日も JOI くんはケーキを焼いて食べようとしたのだが、焼きあがったところで匂いをかぎつけた IOI ちゃんが来たので 2 人でケーキを分けることになった。

ケーキは円形である。ある点から放射状に切り目を入れ、ケーキを N 個のピースに切り分け、ピースに 1 から N まで反時計回りに番号をつけた。つまり、 $1 \leq i \leq N$ に対し、 i 番目のピースは $i-1$ 番目と $i+1$ 番目のピースと隣接している (ただし 0 番目は N 番目、 $N+1$ 番目は 1 番目とみなす)。 i 番目のピースの大きさは A_i だったが、切り方がとても下手だったので A_i はすべて異なる値になった。

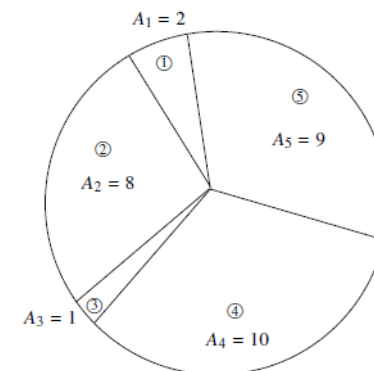
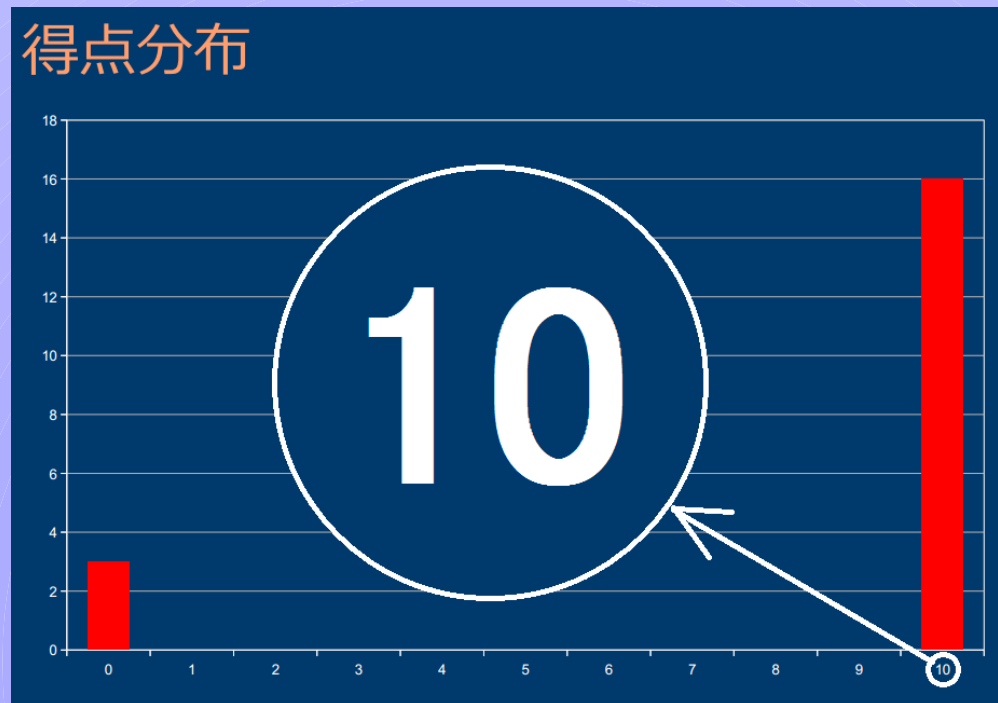


図 1: ケーキの例 ($N = 5, A_1 = 2, A_2 = 8, A_3 = 1, A_4 = 10, A_5 = 9$)

冒頭、完全に一致

- Cake(1): 2013春合宿 Day3
- 得点分布(満点は100です)



で、この問題は…

- 中身は全然違います
- そもそも本選2なのでこんなに難しいはずない
- 見たことある設定だからといって変な読み飛ばしとか誤読とかをしてはいけない

概要

- 環状で数が並んでいる
- JOIくん、IOIちゃんが次のように数を取っていく
 - JOIくんが最初にすきなところを取る
 - IOIちゃんは両端のうち大きいほうを取る
 - JOIくんは両端のうちすきなほうを取る
 - IOIちゃんは両端のうち大きいほうを取る

概要

- JOIくんが最適に操作したときに、JOIくんが取れる数の合計の最大値は？
- $N \leq 2,000$
- IOIちゃんには戦略はありません(残ったものの両端のうち大きいほうをとるだけ)

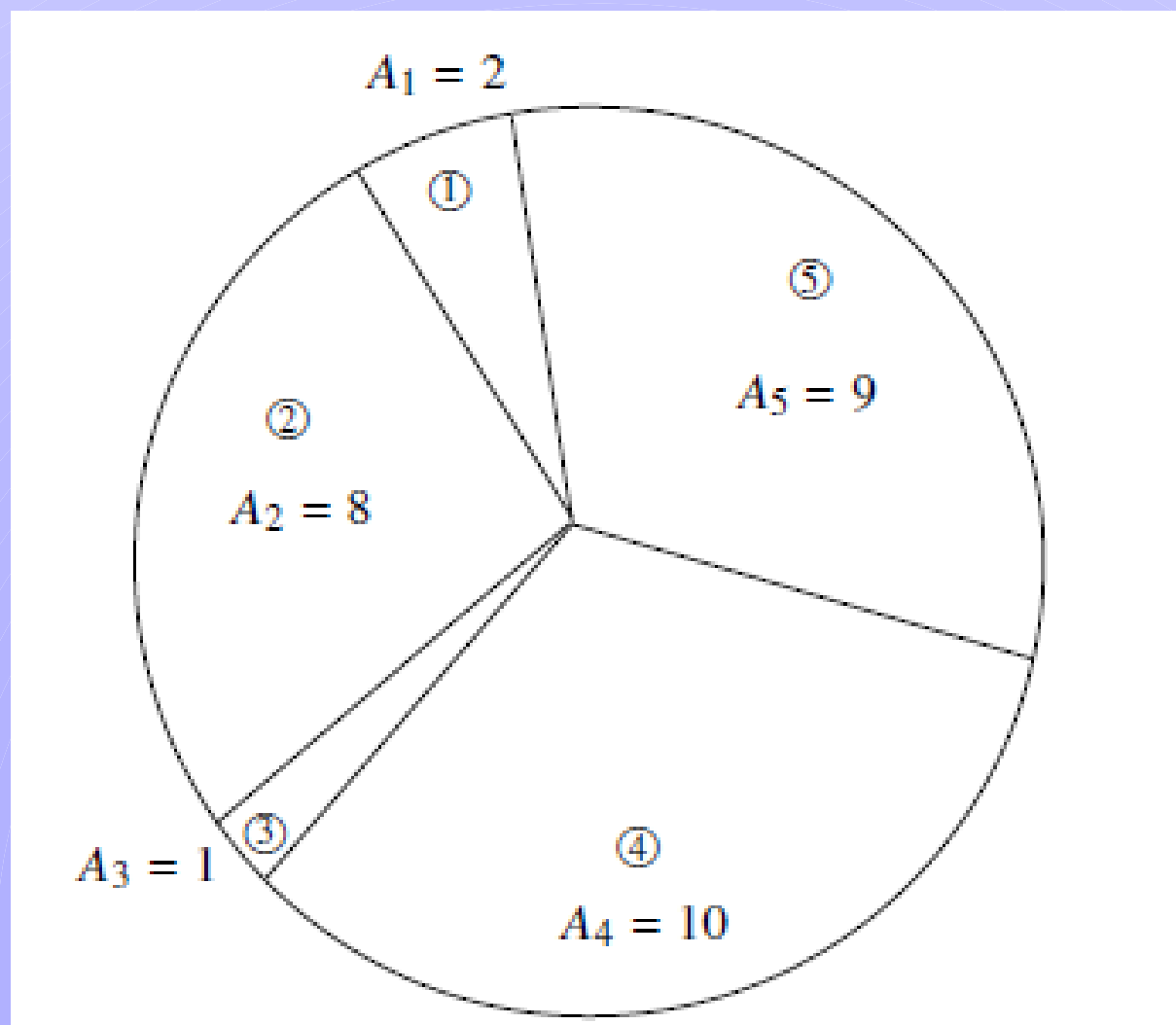
例

JOI KUN

0

IOI CHAN

0



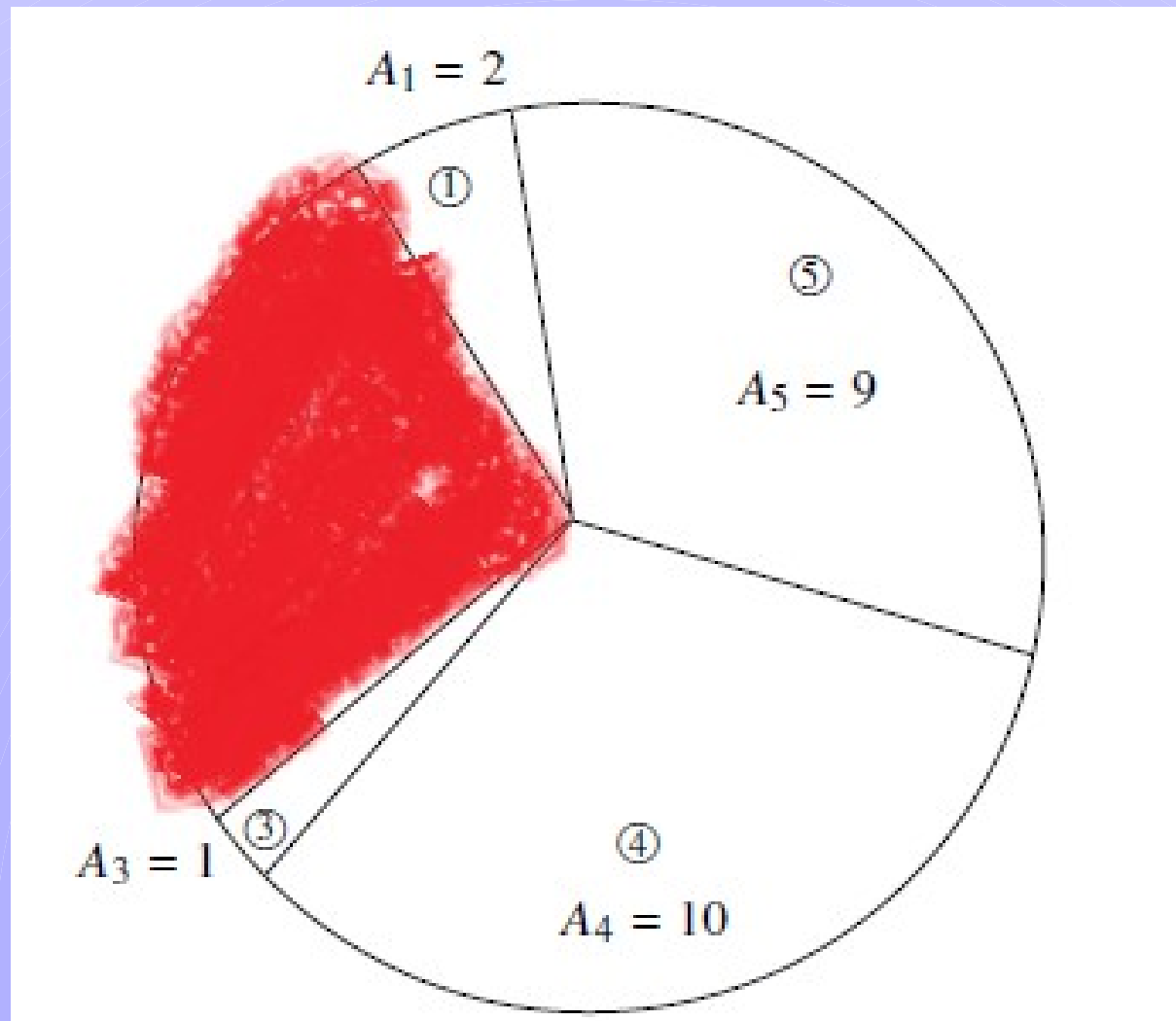
例

JOI KUN

8

IOI CHAN

0



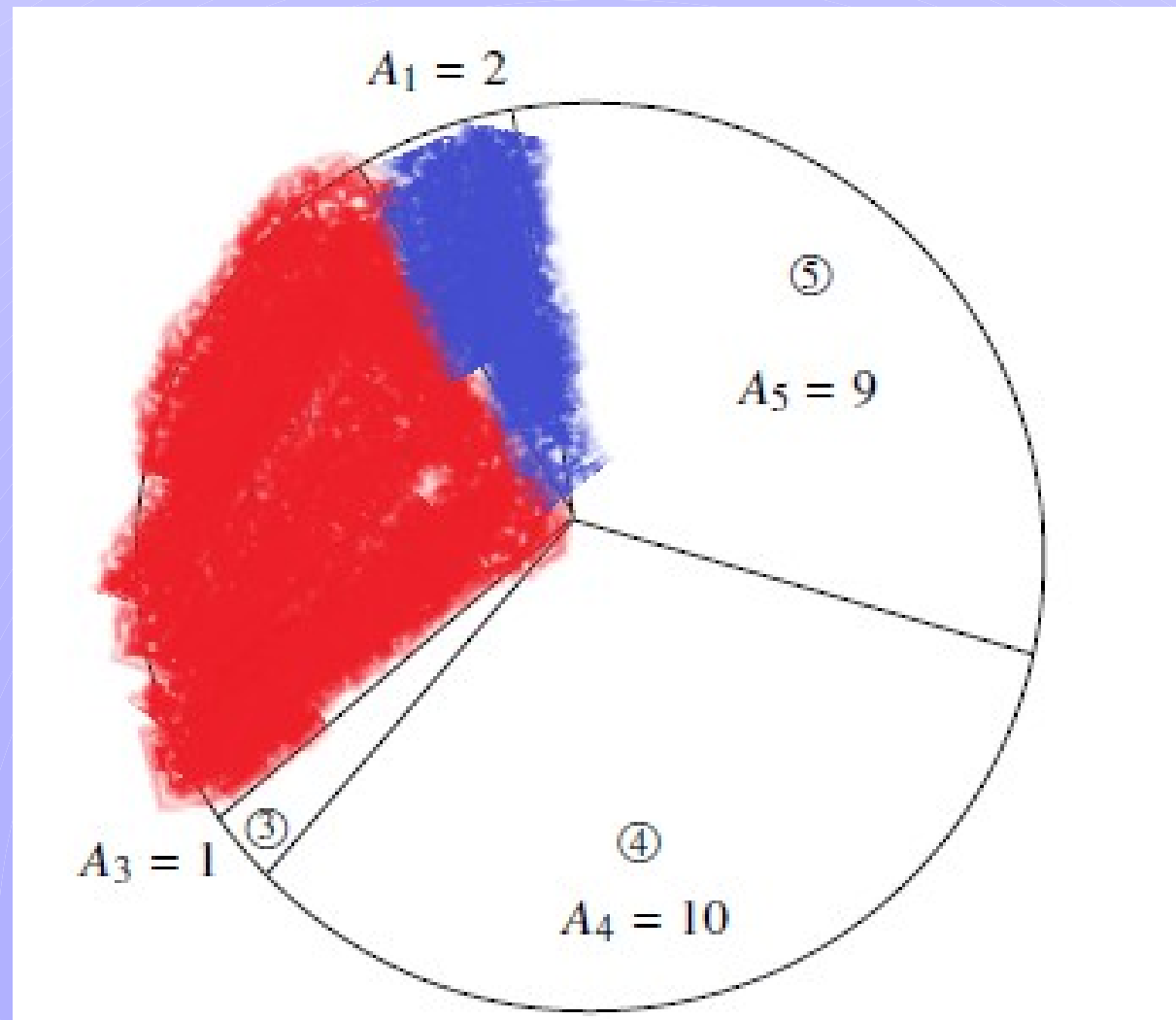
例

JOI KUN

8

IOI CHAN

2



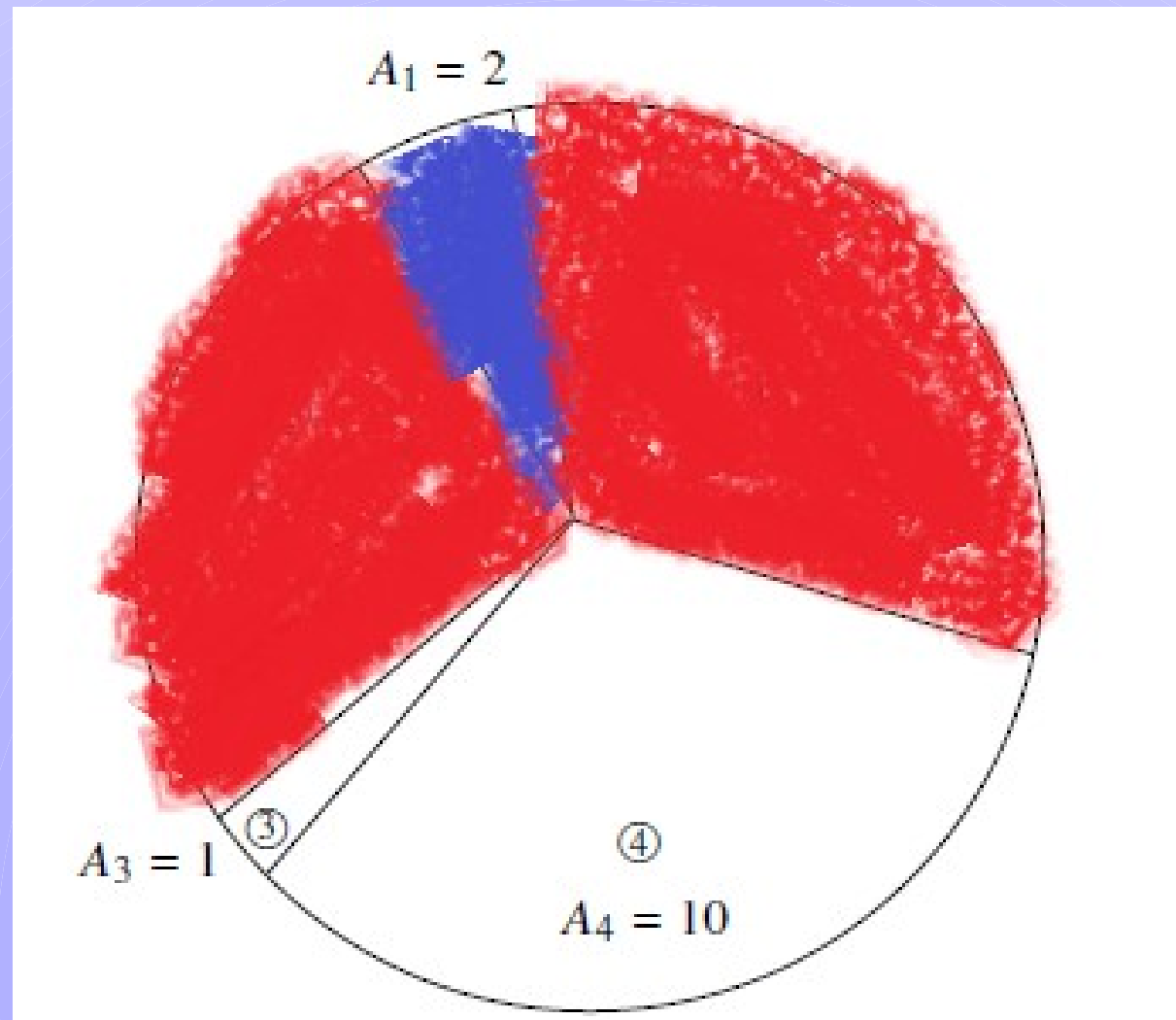
例

JOI KUN

17

IOI CHAN

2



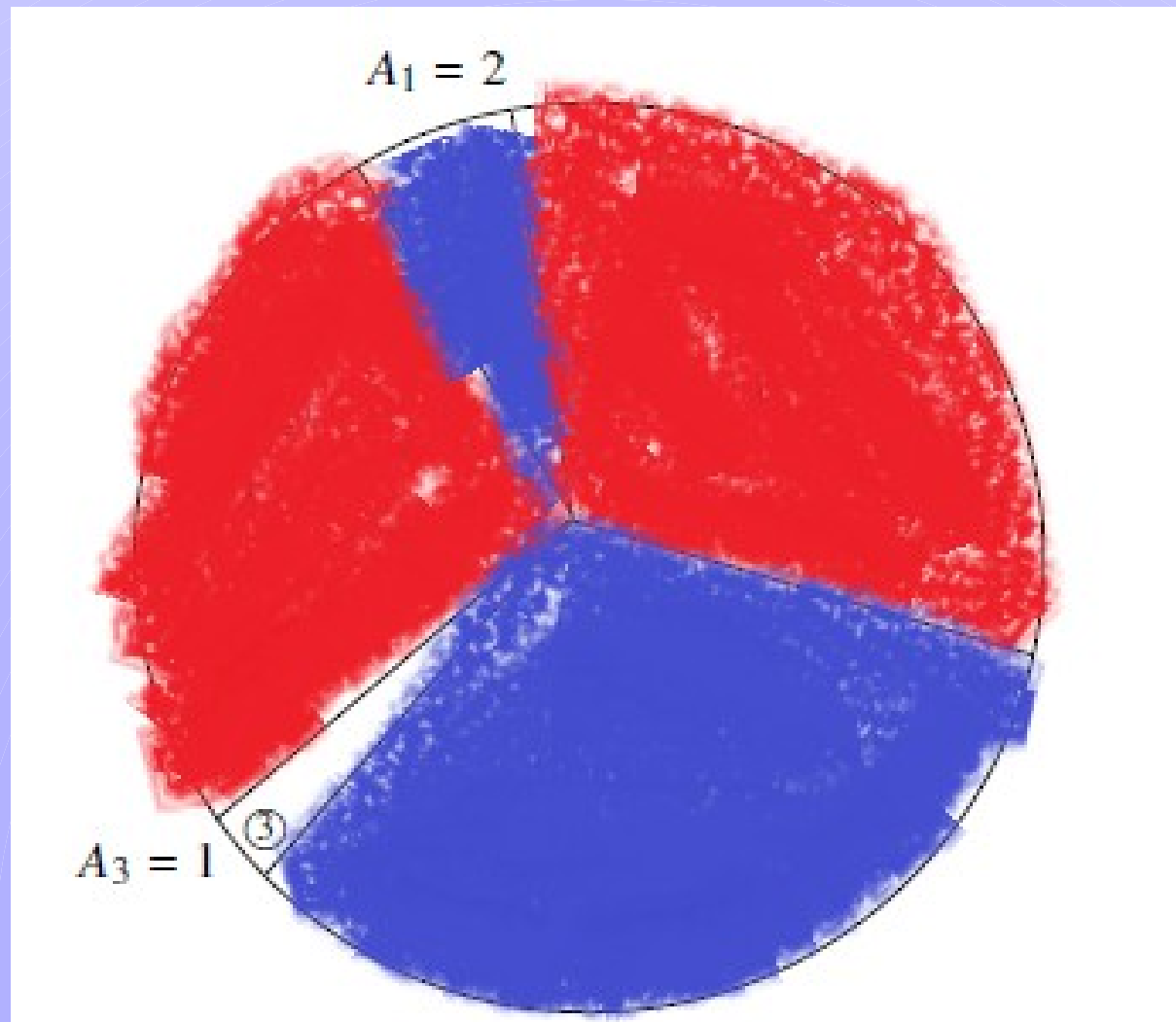
例

JOI KUN

17

IOI CHAN

12



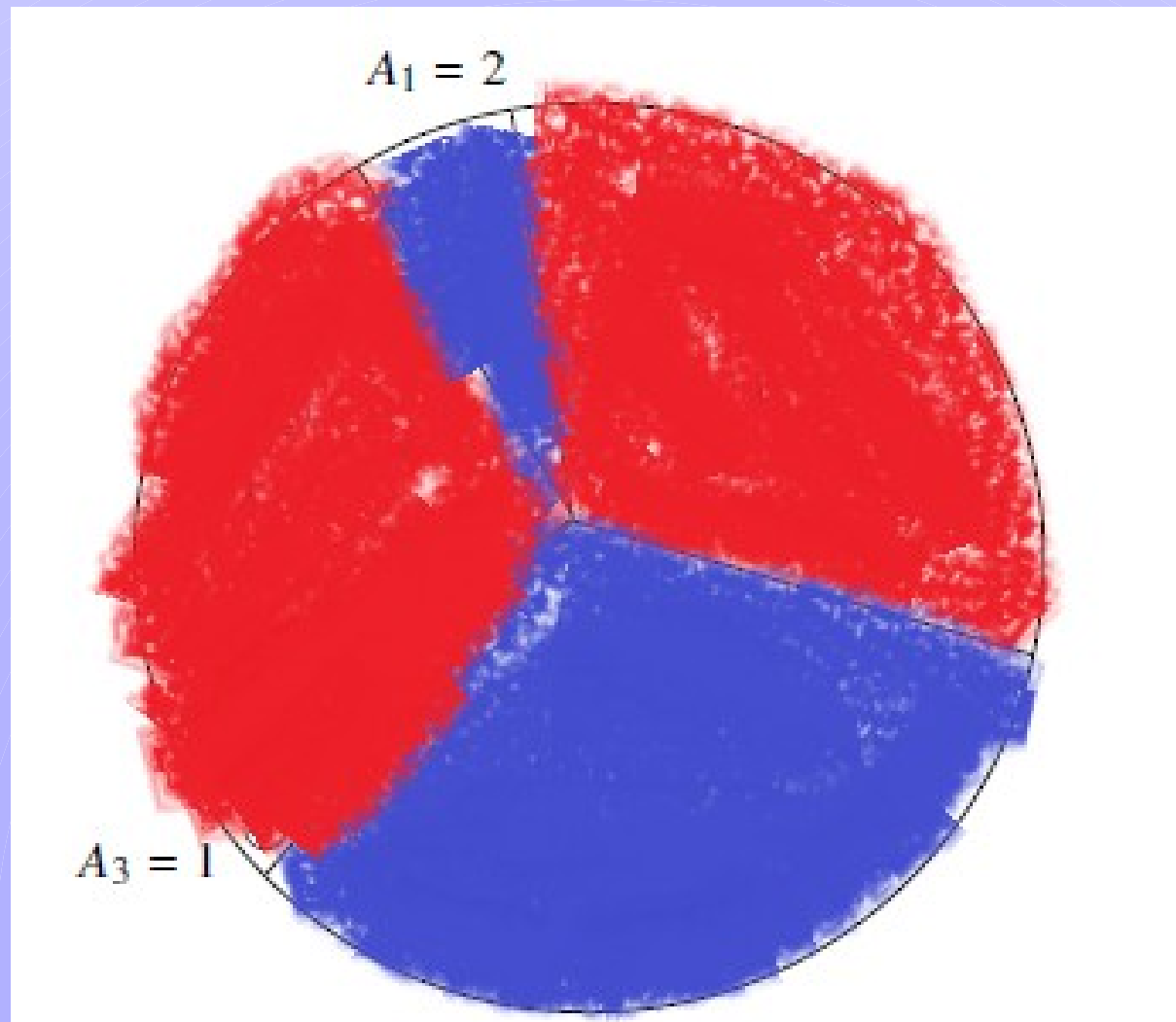
例

JOI KUN

18

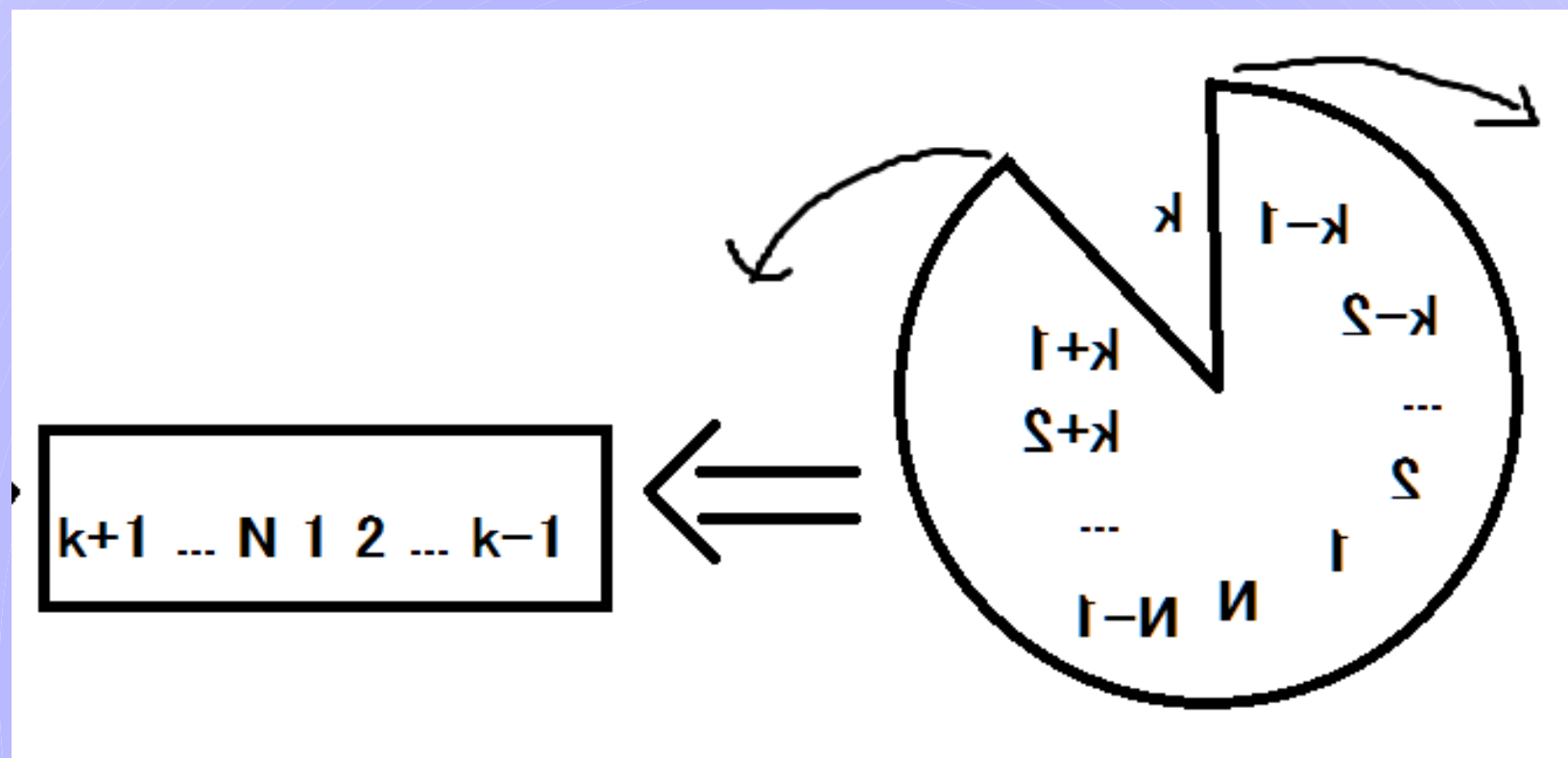
IOI CHAN

12



とりあえず円環は複雑なので、

- JOIくんが最初にとるケーキを全探索
- 残りのケーキは一直線上におけるとしてよい



小課題1

- JOIくんが両方のケーキを取れるときにどっちを取ったかを全探索
- JOIくんがケーキを取るのは $N/2$ 回程度
- IOIちゃんの選択は分岐しない(大きいほう)
- 計算量 $O(2^{N/2})$ くらい 15点

状態をまとめる

- 全探索は無駄が多い
- たとえば、 $[i,j]$ の外側でどんな風を取っていても「残り $[i,j]$ 」の状況からの取り方には影響しない

状態をまとめる

- 全探索は無駄が多い
- たとえば、 $[i,j]$ の外側でどんな風を取っていても「残り $[i,j]$ 」の状況からの取り方には影響しない
- 動的計画法が使える好条件では？

状態をまとめる

- 全探索は無駄が多い
- たとえば、 $[i,j]$ の外側でどんな風を取っていても「残り $[i,j]$ 」の状況からの取り方には影響しない
- 動的計画法が使える好条件では？
- というかあの図を見たら区間DPっぽいよね
- 典型

動的計画法パート

- $dp[i][j]$:= 残り $[i,j]$ の状況からはじめてときのJOIくんの取り分の最大値
- 「残り $[i,j]$ の状況からはじめてとき」がJOIくんとIOIちゃんどっちのターンから始まるかは、区間のながさの偶奇でできる(ので一意)
- なので $dp[i][j][0]$, $dp[i][j][1]$ に分ける必要はない

動的計画法パート

- 更新はこんな感じの式
- JOIくんのターンから始まる場合

$dp[i][j]=$

$\max(dp[i+1][j]+A[i], dp[i][j-1]+A[j]);$

- IOIちゃんのターンから始まる場合

$dp[i][j]=dp[i+1][j]$ ($A[i]>A[j]$ のとき)

$dp[i][j]=dp[i][j-1]$ ($A[i]<A[j]$ のとき)

小課題2 まとめ

- JOIくんの最初にとるものを決める($O(N)$)
 - 状態数 $O(N^2)$ のDPをする
 - (メモ化再帰で書くと楽)
-
- 全部で $O(N^3)$ 55点

満点解法

- ここまできたらもう余興
- JOIくんの最初の選び方ごとくにわざわざいままでのメモしたものを消す必要はない
- ので、状態数は合計で $O(N^2)$
- 計算量も $O(N^2)$
- 100点

円環

- 円環は実装が結構ややこしいと言われます



なんか出前に似てるすぬ系だそうだよ。 @snuke_ · 8月12日
円環お断り

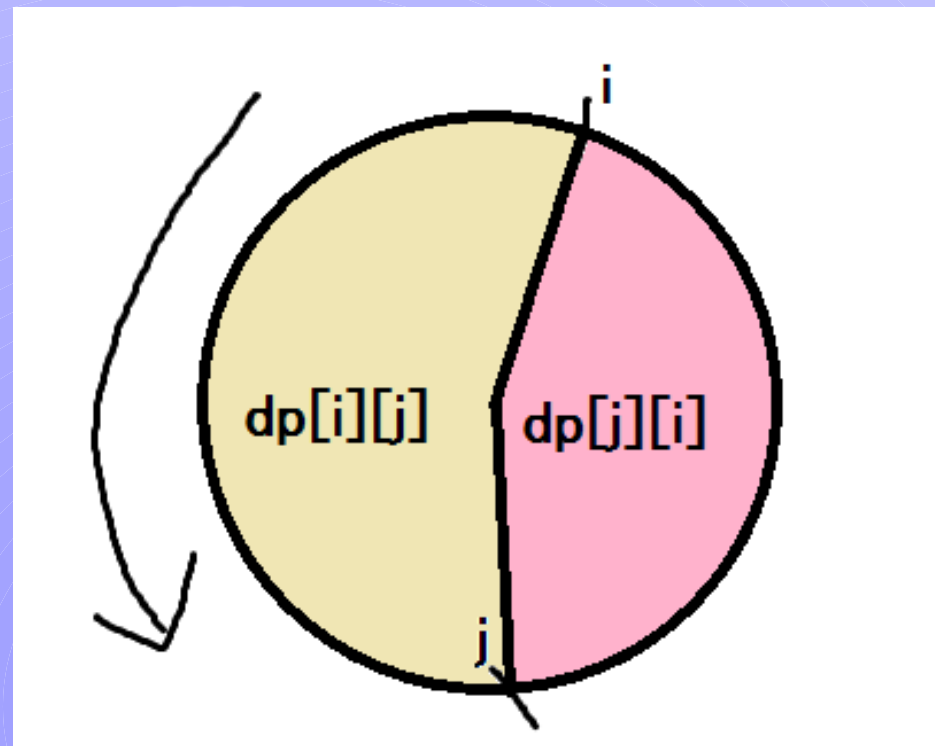


なんか出前に似てるすぬ系だそうだよ。 @snuke_ · 8月12日
二回連続で円環系のMedium落としてる。円環は闇。



実装法[1]

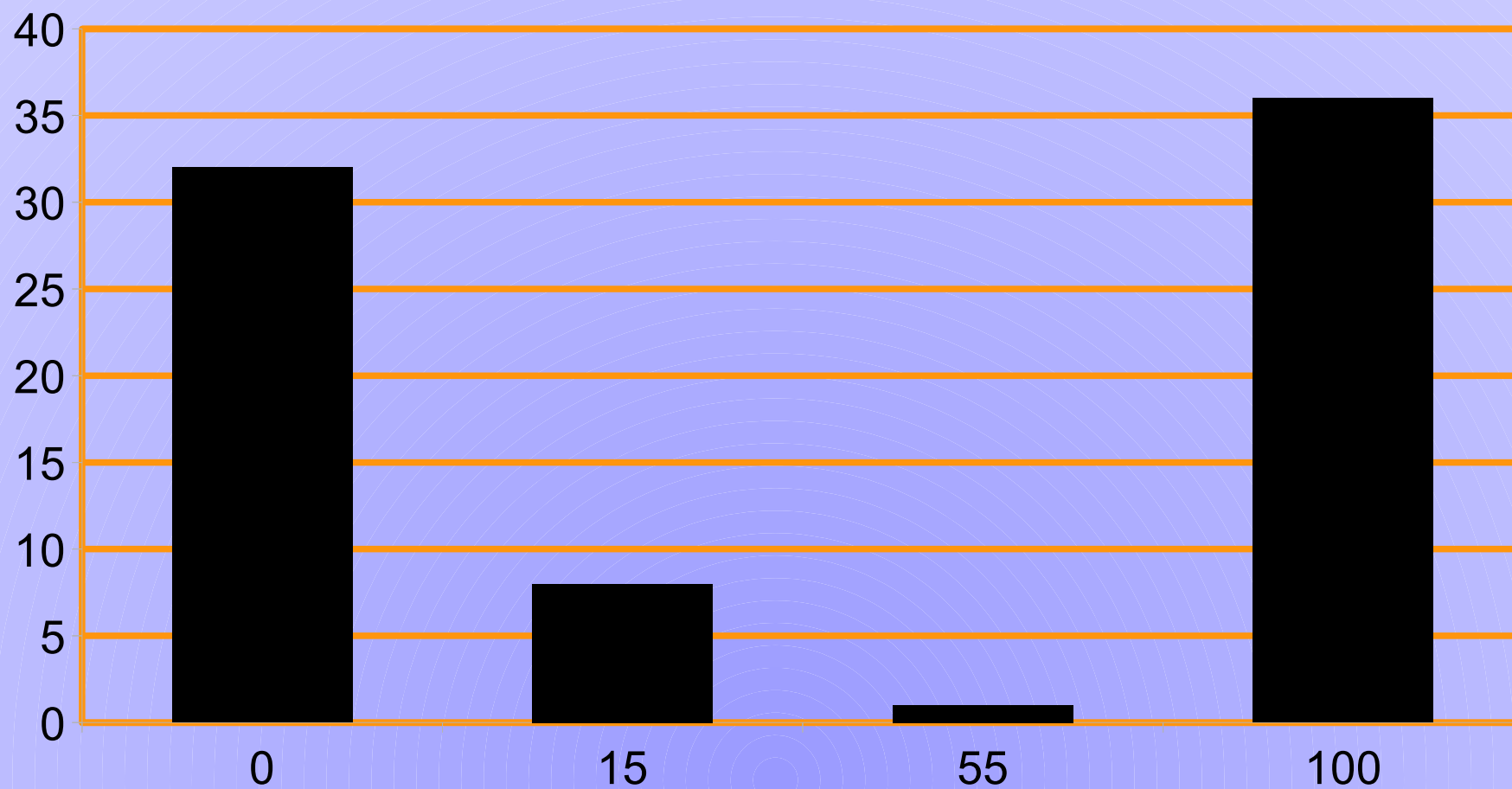
- 本当に $dp[i][j]$ は*i*から始まって*j*で終わる区間として、そのまま持つ。
- こんなイメージ



実装法[2]

- 1番目からN番目を一列に順に2回ならべる
- そうすると円環を区間につぶしたまま作業できる
- $dp[1][N-1], dp[2][N], dp[3][N+1], \dots, dp[N][N+N-2]$
を計算していくことになる
- 定数に2や4がかかるけど間に合う

得点分布



第14回 日本情報オリンピック 本選

問題3 JOI公園(JOI Park) 解説

二階堂建人



問題概要

広場が N 個ある公園があり、広場どうしは M 本の道でつながっている。道 i は広場 A_i と 広場 B_i をつないでおり、長さは D_i である。

まず、値 X を決め、広場 1 から距離 X 以内の広場どうしを全て地下道でつなぐ。これにはコストが $C \cdot X$ かかる。

次に、地下道でつながれている広場どうしをつないでいる道をなくす。

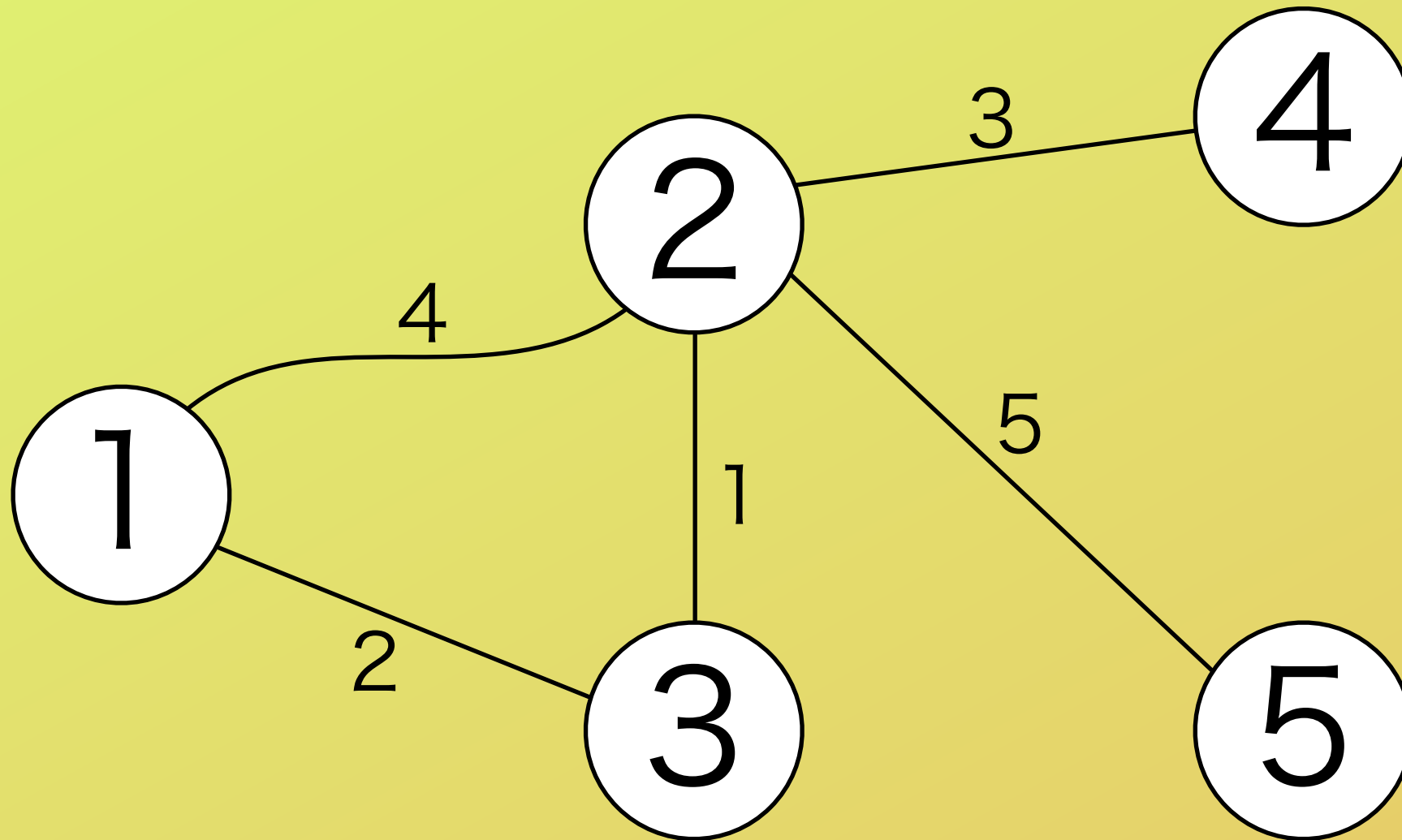
最後に、残った道を補修する。道 i を補修するときにかかるコストは D_i である。

かかるコストの和の最小値を求めよ。

サンプル

入出力例 1

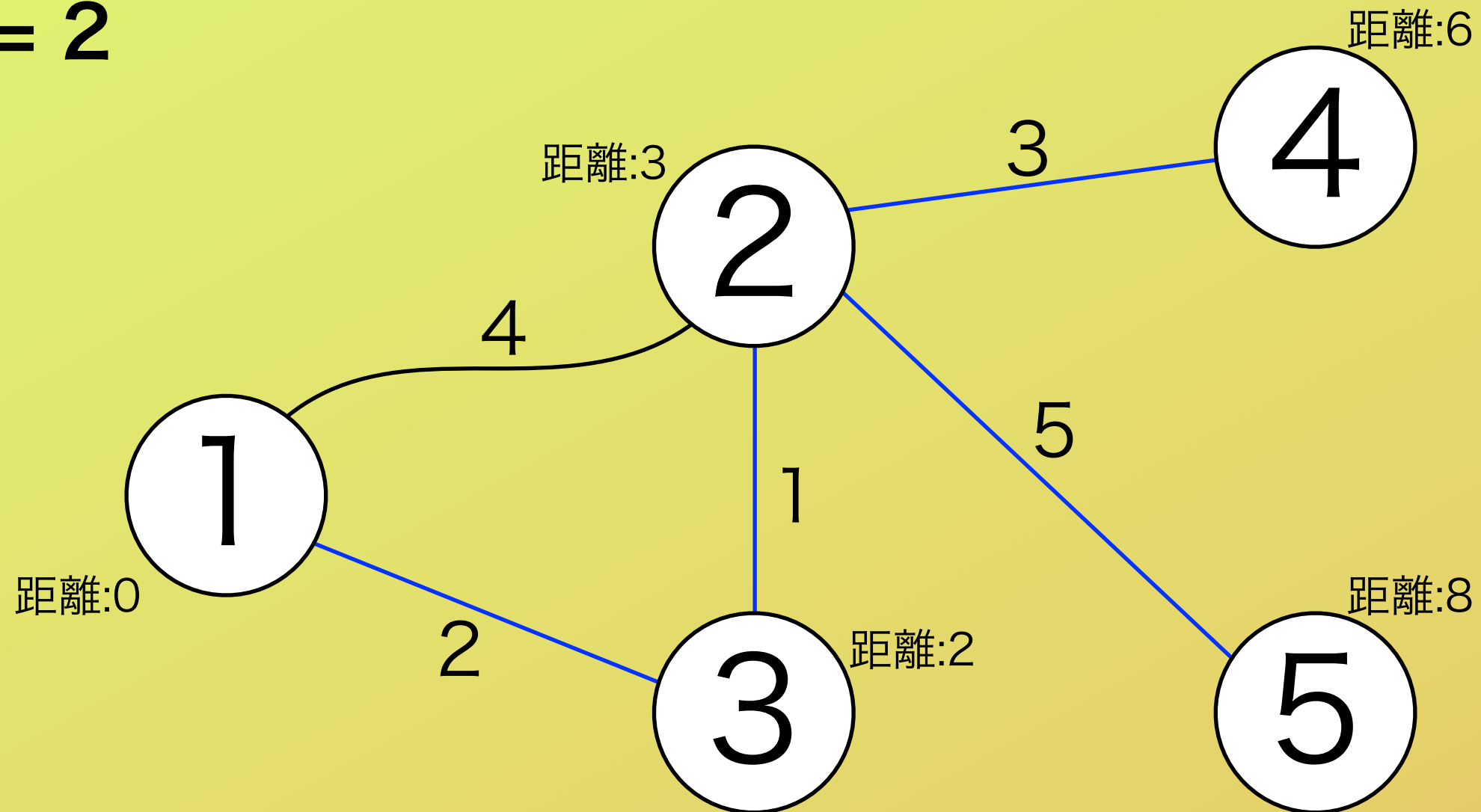
$C = 2$



サンプル

入出力例 1

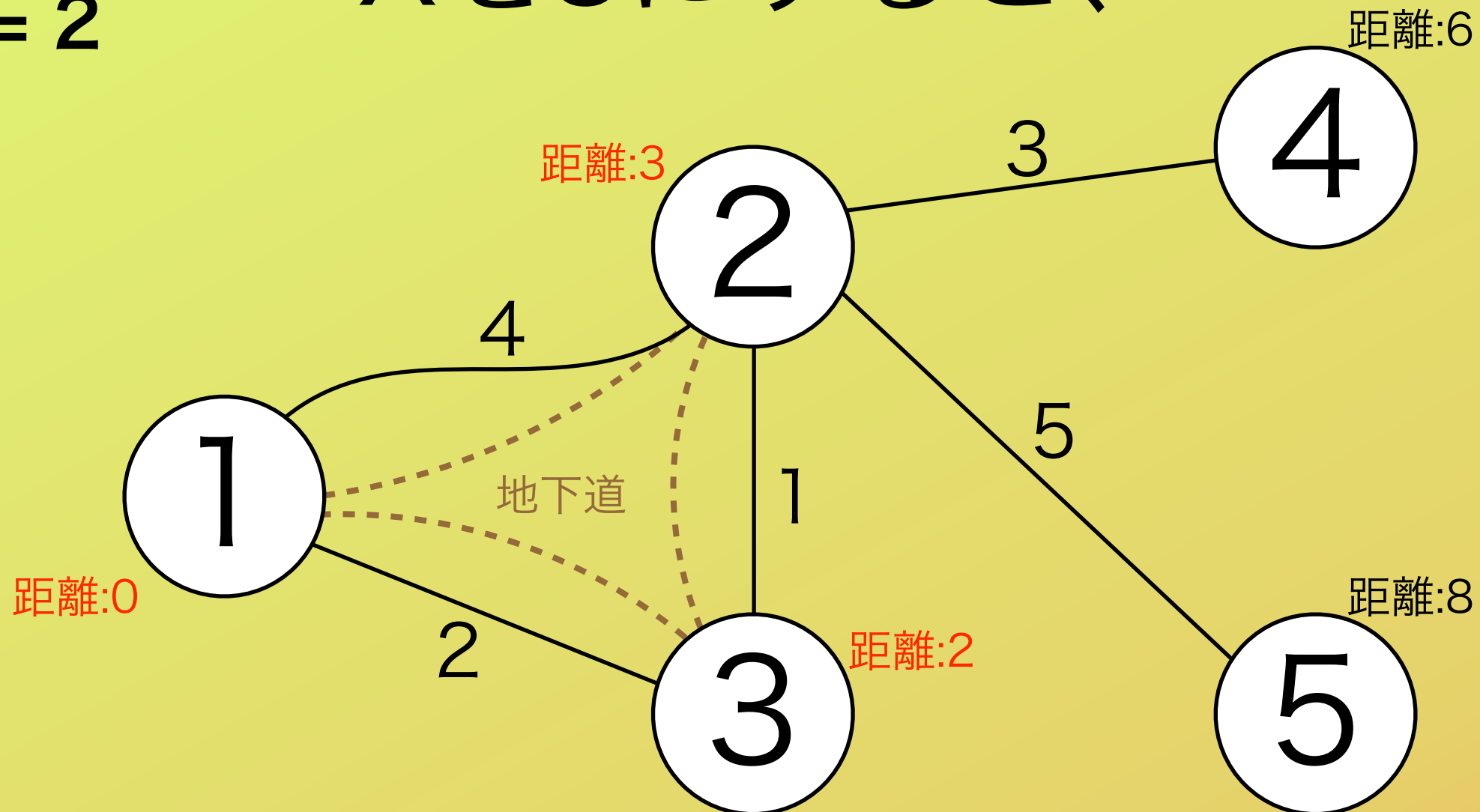
$C = 2$



サンプル

入出力例 1
 $C = 2$

Xを3にすると、



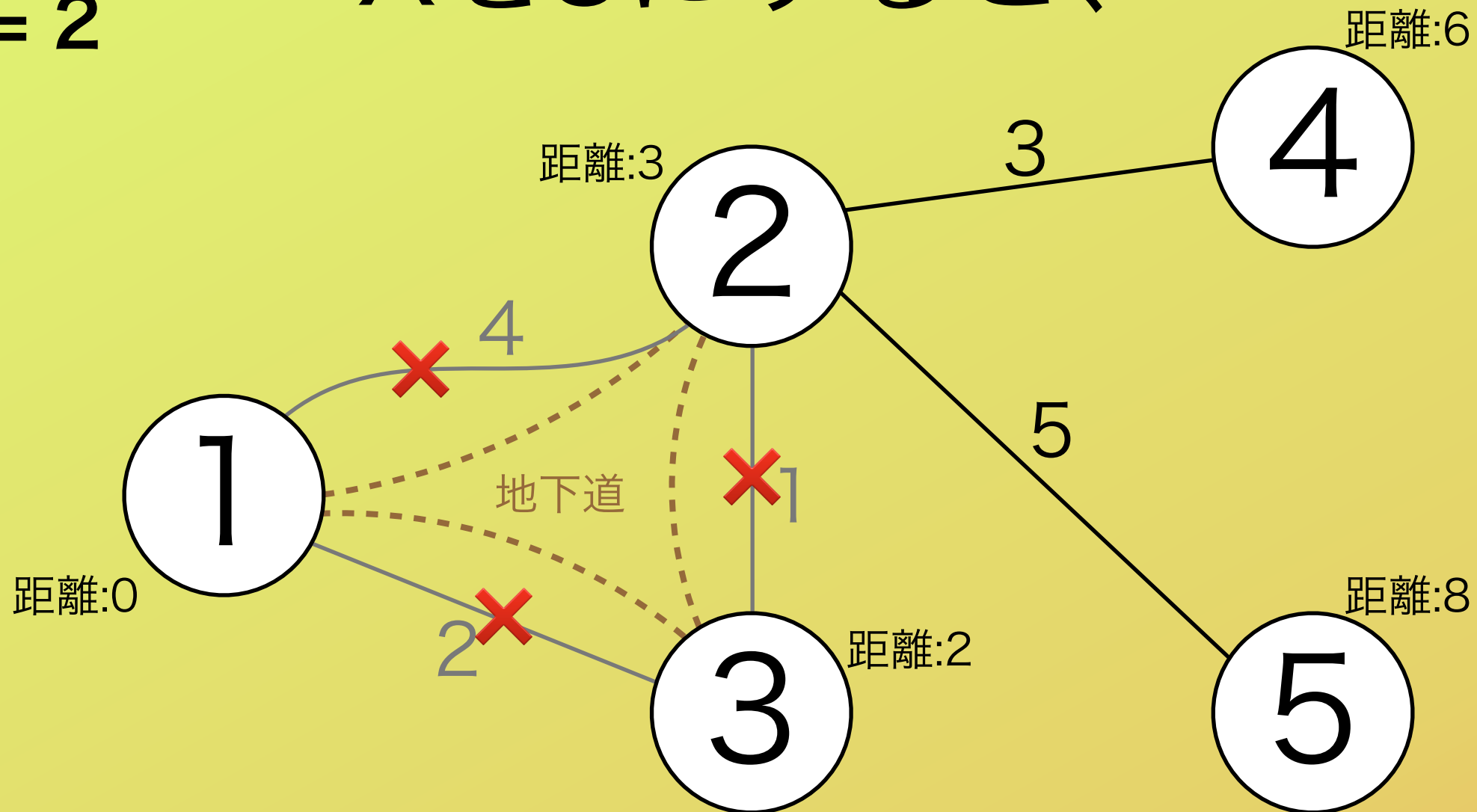
まず、広場1との距離が3以内の広場どうしを地下道でつなぐ。

このとき、コストは $C * X = 2 * 3 = 6$ かかる。

サンプル

入出力例 1
 $C = 2$

Xを3にすると、

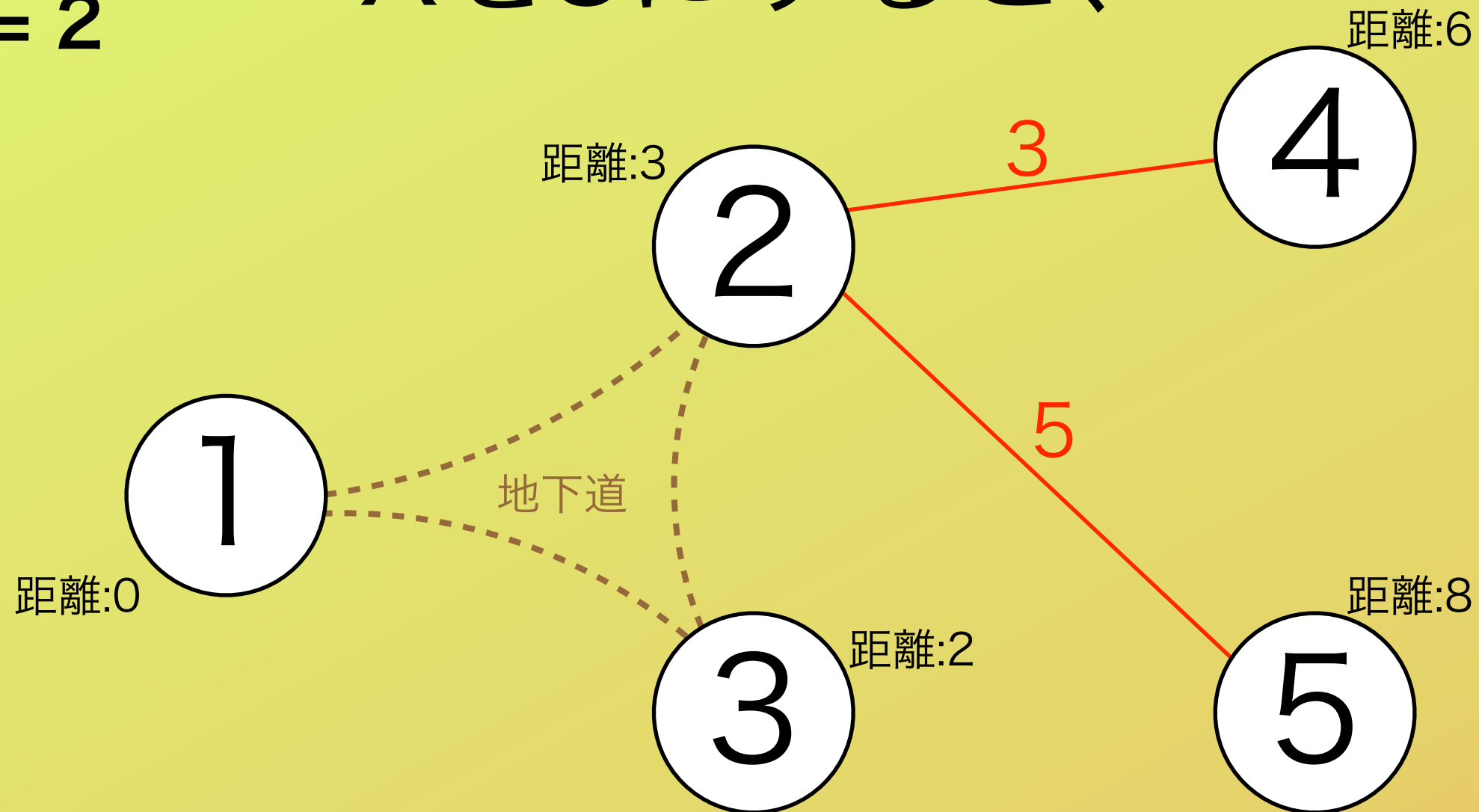


次に、地下道でつながれた広場どうしをつなぐ道をなくす。

サンプル

入出力例 1
 $C = 2$

Xを3にすると、



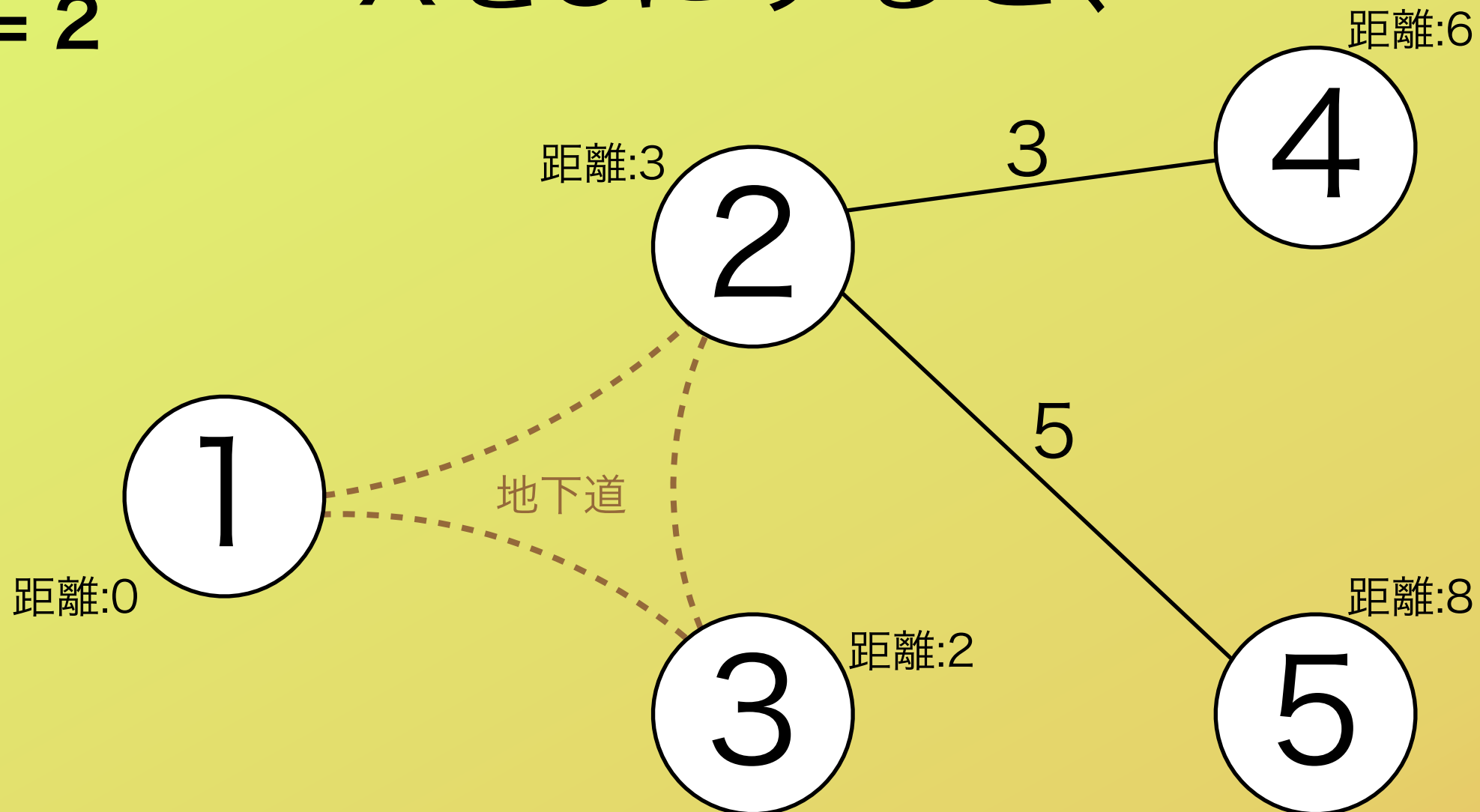
最後に、残った道を補修する。

このとき、コストは補修する道の長さの和 $= 3+5 = 8$ かかる。

サンプル

入出力例 1
 $C = 2$

Xを3にすると、



地下道のコストが 6、道の補修のコストが 8 かったので、
コストの和は 14 となる。

距離の求め方

まずは、広場 1 からの距離を求める必要がある。

広場を頂点、道を辺としたグラフの

最短経路問題

を解けばよい。

距離の求め方

最短経路問題を解くアルゴリズムには、

- Dijkstra 法（ダイクストラ）
- Bellman-Ford 法（ベルマンフォード）
- Warshall-Floyd 法（ワーシャルフロイド）

等がある。

これらのアルゴリズムは書籍・WEB共に文献がたくさんあるので、もし知らなかった人は各自で調べましょう。

書籍の例：プログラミングコンテストチャレンジブック（蟻本）

距離の求め方

それぞれのアルゴリズムの計算量は、

- Dijkstra 法 : $O(M \log N)$ (実装によりますが)
- Bellman-Ford 法 : $O(NM)$
- Warshall-Floyd 法 : $O(N^3)$

小課題 1 では $N \leq 100$, $M \leq 200$ で、小課題 2 では $N \leq 100$, $M \leq 4000$ なので、どの方法を使っても間に合う。

小課題 3 では $N \leq 100000$, $M \leq 200000$ なので、Dijkstra 法を使わないと間に合わない。

小課題1(15点)

制約

- $N \leq 100$
- $M \leq 200$
- $C \leq 100$
- $D_i \leq 10$

D_i (道の長さ) がとても小さい。

小課題1(15点)

X の値として意味のあるものを考える。

- ・ 広場 1 から最も遠い広場までの距離より大きい X は試す意味がない。

→ それより大きくしても地下道は増えない。

$N \leq 100$, $D_i \leq 10$ なので、広場 1 から最も遠い広場までの距離は高々 1000 程度。

小課題1(15点)

X を 0 から 1000 まで試し、それぞれの場合にかかるコストのうちの最小値を求めればよい。

かかるコストは素直に、地下道でつながるような広場を調べ、補修しなければならない道を調べればよい。

計算量は、 $O(\text{試す}X\text{の個数} * (N+M))$ (+最短路の計算量) であり、間に合う。

ちなみに、計算量を表す時の“+”はmaxのような意味を表します。

例えば、 $O(N+M) = O(\max(N,M))$

小課題2(45点)

制約

- $N \leq 100$
- $M \leq 4000$
- $C \leq 100000$
- $D_i \leq 100000$

D_i (道の長さ) が大きい。

小課題2(45点)

X の値として意味のあるものをさらに考える。

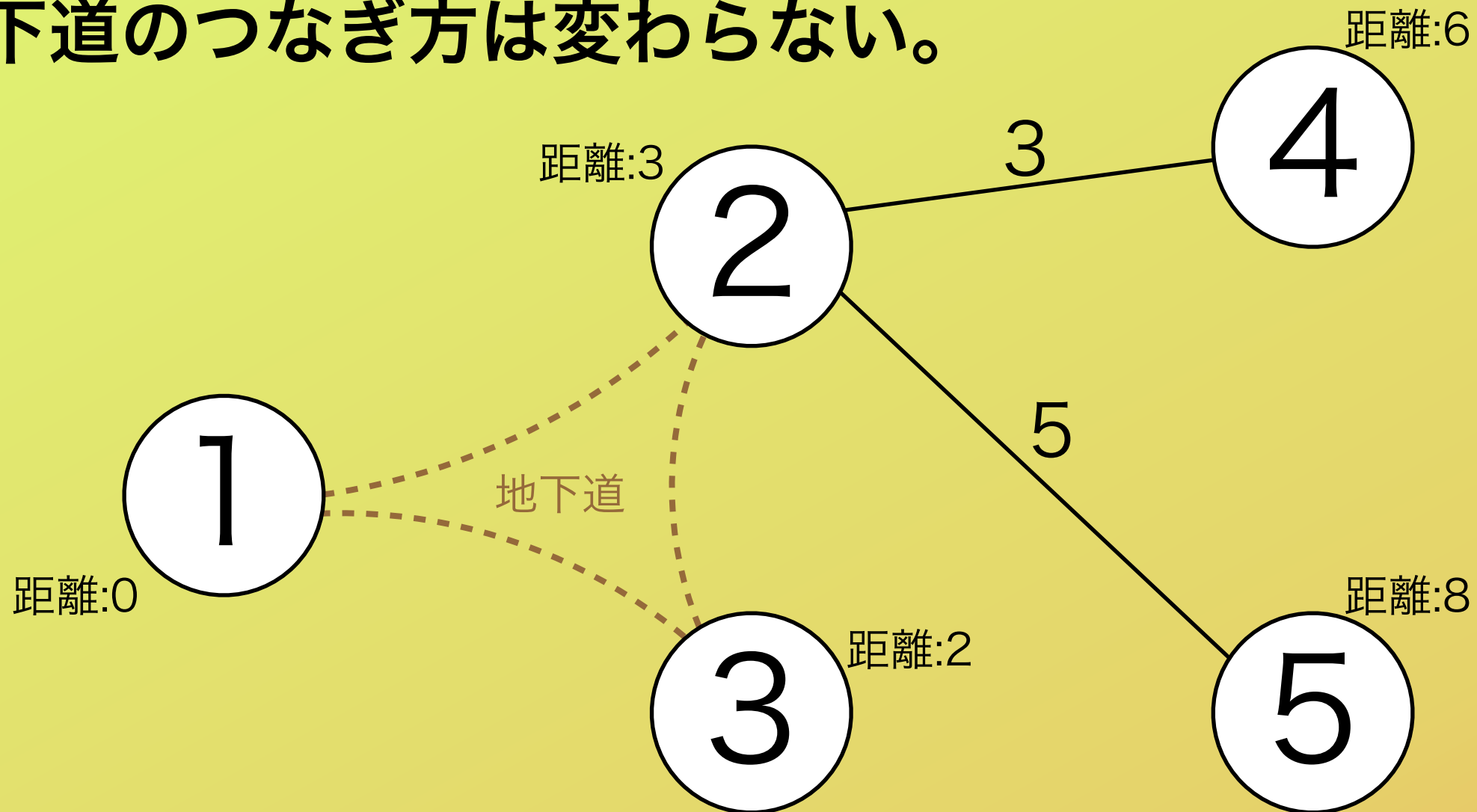
異なるいくつかの値で地下道のつながり方が同じならば、
その中で最も小さい値のみを試せばよい。

→ ちょうど地下道が増えるようなぎりぎりの値のみ
を試せばよい。

→ 広場 1 からある広場までの距離の値のみを X とし
て試せばよい。

サンプル

X を 3 にした場合と 4 や 5 にした場合では、
地下道のつながり方は変わらない。



小課題2(45点)

広場 1 から各広場までの距離の値をそれぞれ X の値として試し、それぞれの場合にかかるコストの最小値を求めればよい。

かかるコストは小課題 1 と同様にして調べればよい。

X の候補は高々 N 通り。

計算量は、 $O(N * (N+M))$ (+最短路の計算量)

であり、間に合う。

小課題3(40点)

制約

- $N \leq 100000$
- $M \leq 200000$
- $C \leq 100000$
- $D_i \leq 100000$

N,M が大きい。

小課題3(40点)

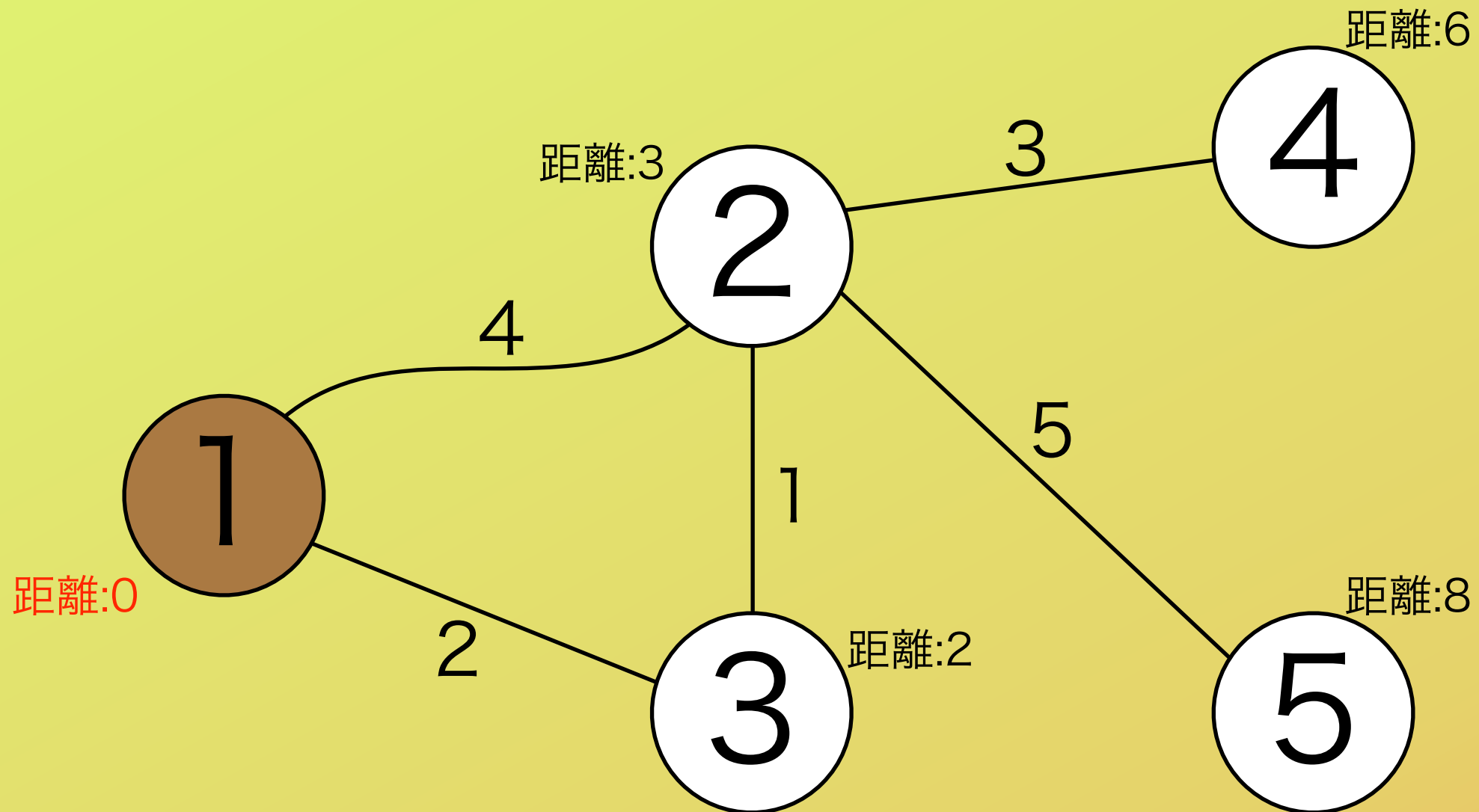
X の値が増えていくにつれ、地下道の本数は増えていき、補修する必要のある道の本数は減っていく。

言い直すと、 X の値を小さい方から試していくとき、補修する必要のある道の本数は単調に減少する。

X の値を増やすたびに、それによって補修する必要の無くなった道を消していくことにする。

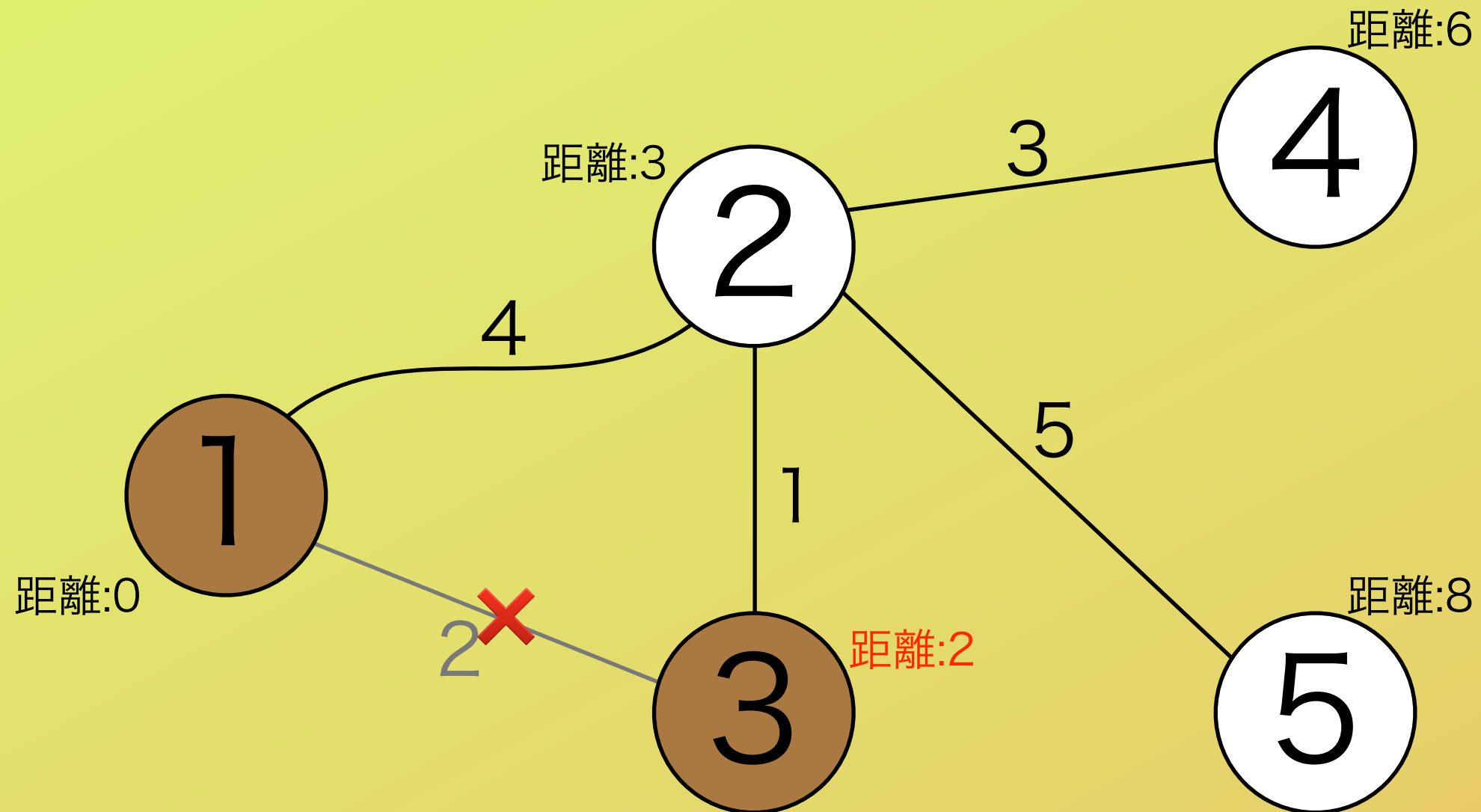
サンプル

$X = 0$



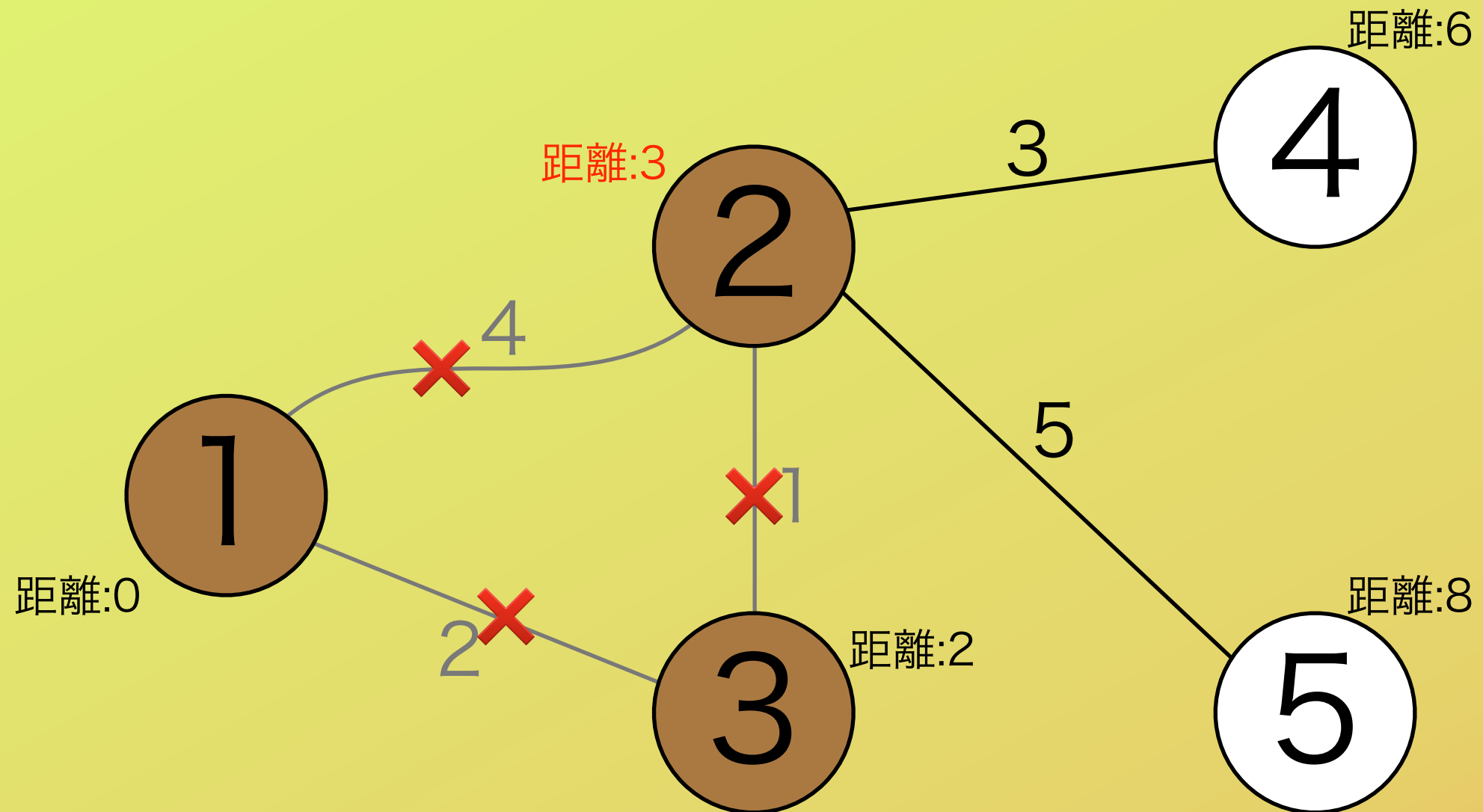
サンプル

$X = 2$



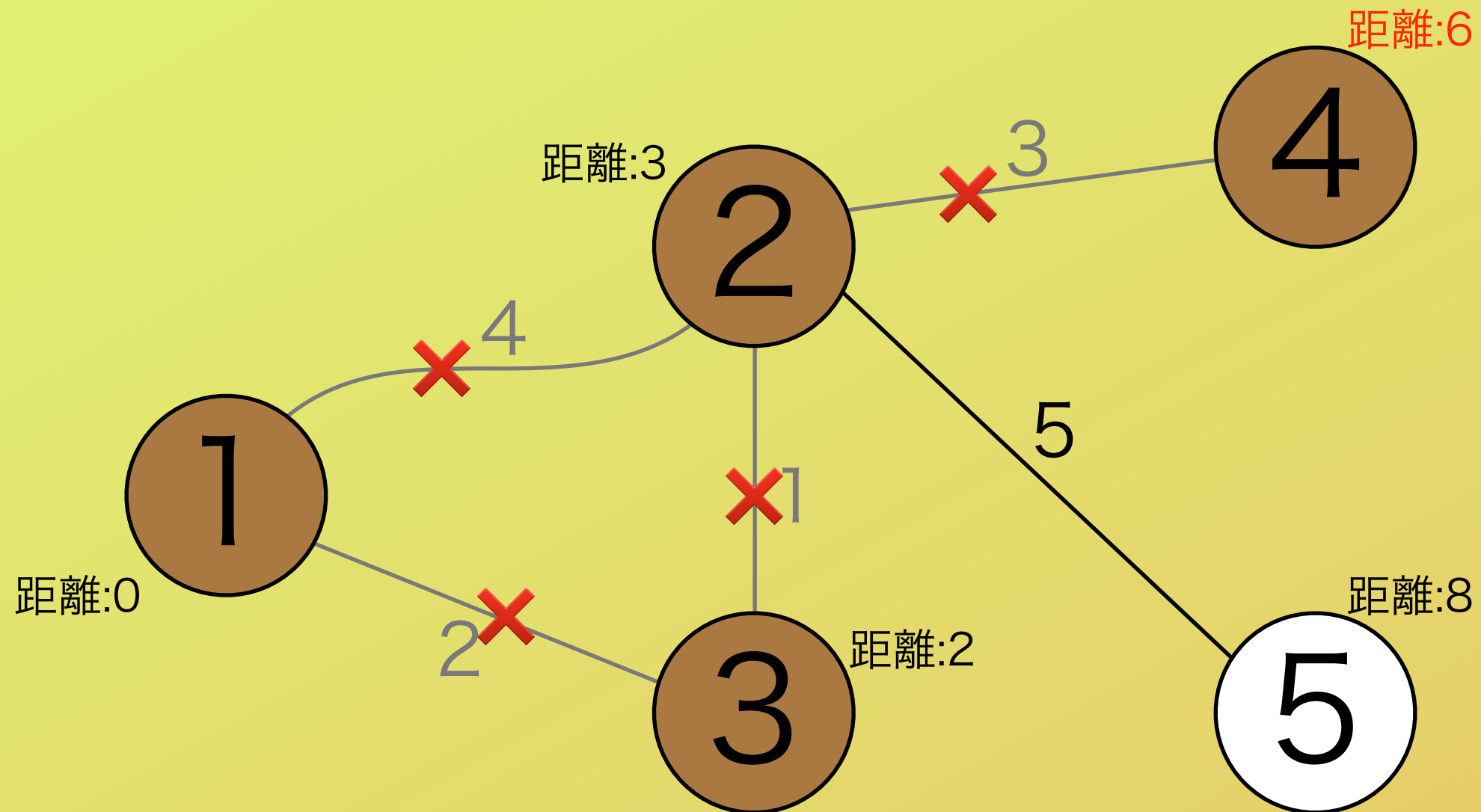
サンプル

$X = 3$



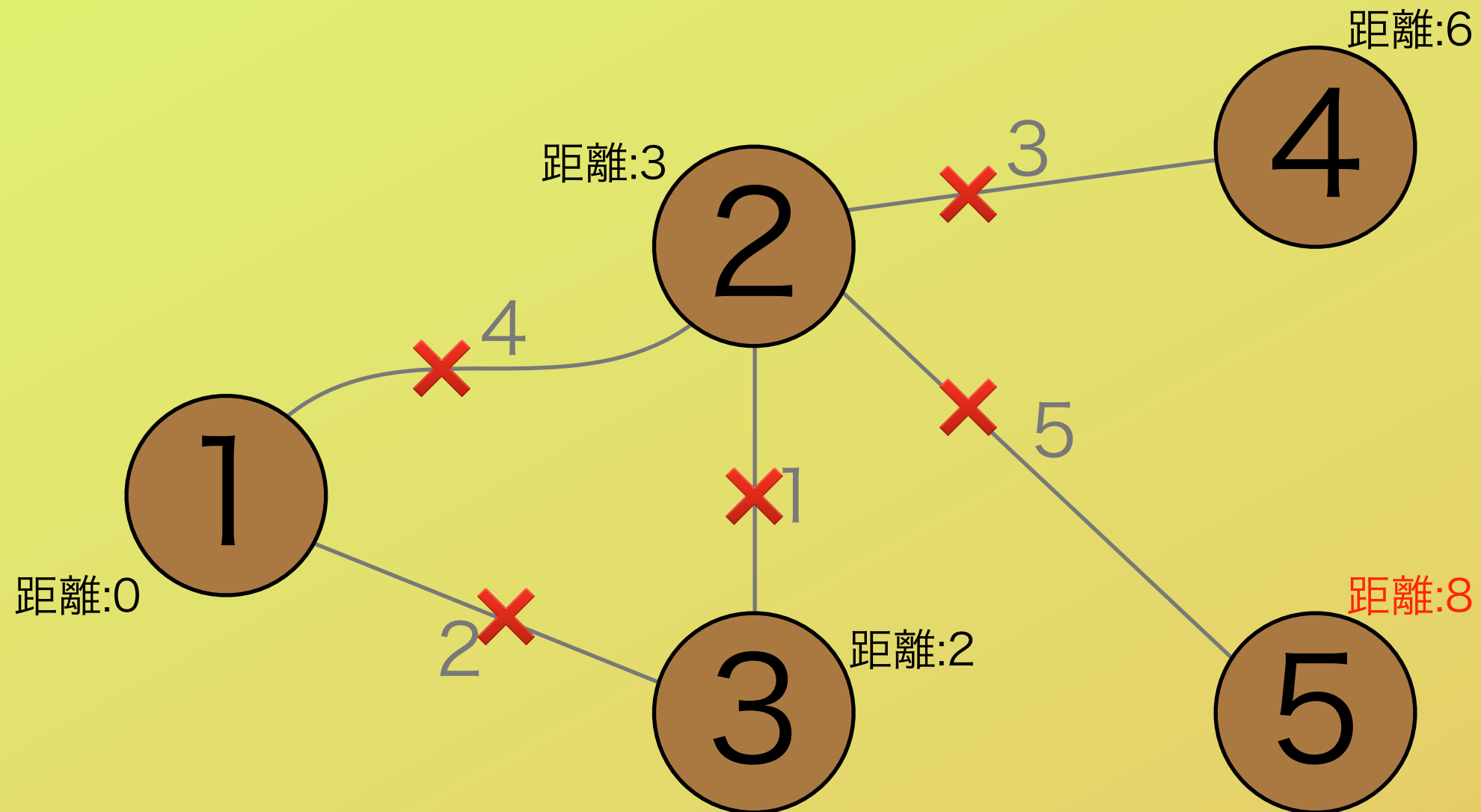
サンプル

$X = 6$



サンプル

$X = 8$



小課題3(40点)

X の値として広場 1 から広場 i までの距離の値を試すとき、新たに補修する必要がなくなる道は？

- ・ 広場 i に直接つながっている道
- ・ かつ、相手の広場が既に地下道でつながっている広場であるような道

ということは X の値を試すごとに、広場 i に直接つながっている道それぞれについて、「相手の広場が既に地下道でつながっているかどうかを調べ、つながっているなら道をなくし、そうでないなら何もしない」という操作をすれば良い。

小課題3(40点)

計算量は、

$O(N + \text{「各広場につながっている道の本数の和」})$

であるが、それぞれの道がつながっている広場が2個ずつであることを考えると「各広場につながっている道の本数の和」は「道の本数 $\times 2$ 」に等しいため、

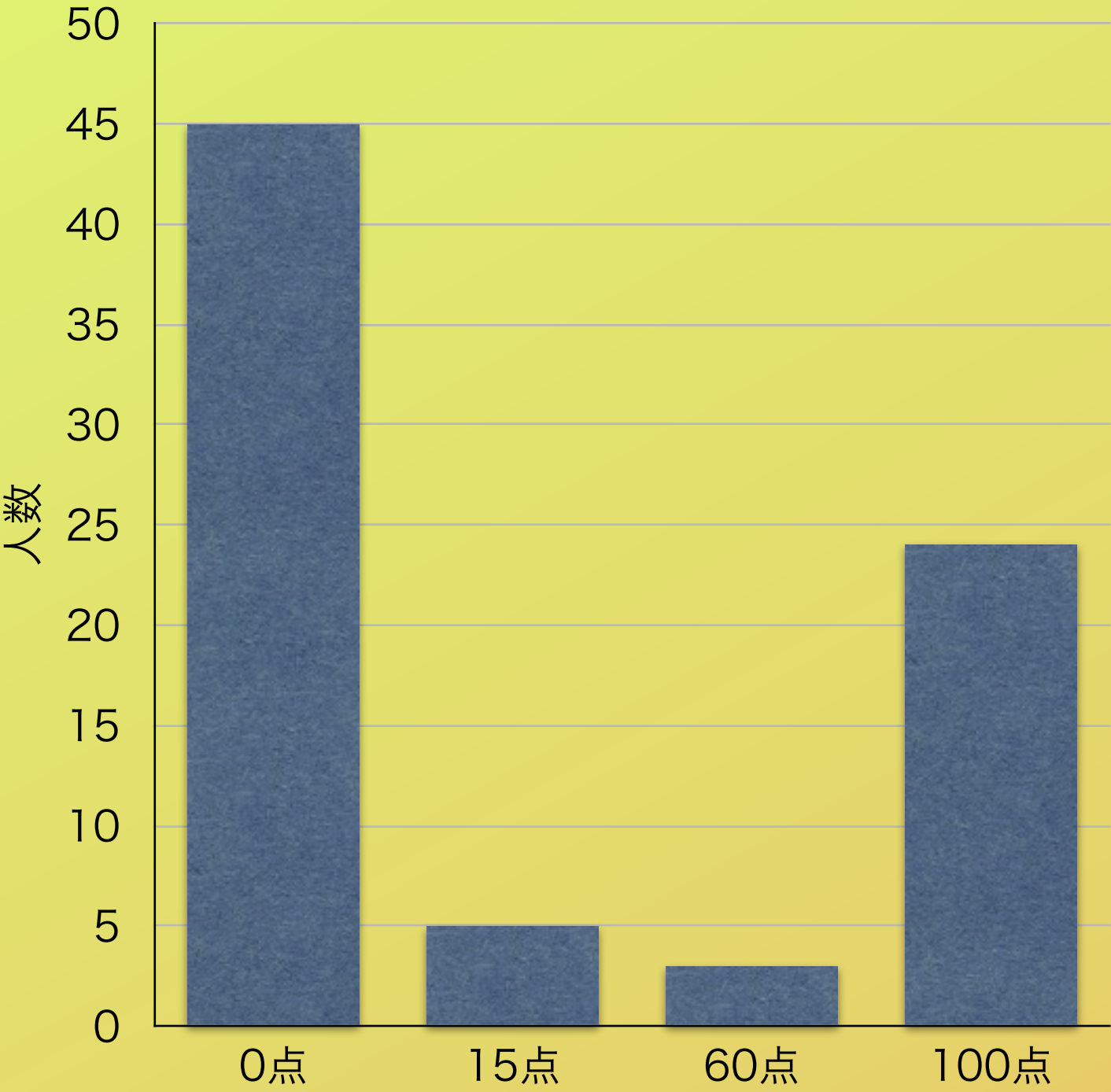
$O(N + M)$

となる。試す X の値をソートすることとも考慮すると、

$O((N \log N) + N + M)$ (+最短路の計算量)

となり、最短路を求めるアルゴリズムにDijkstra法を使っていけば間に合う。

得点分布



第 14 回情報オリンピック本選

問題 4 – 舞踏会(Ball)

解説：城下慎也(@phidnight)

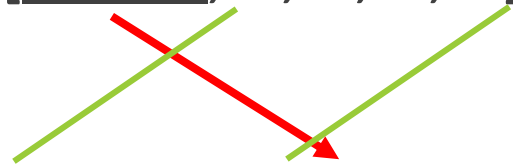
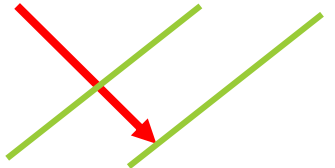
問題概要

- N 人の貴族が一行に並ぶ。
 - 既に M 人の貴族については、初期状態で並ぶ位置が決まっている。
 - 「先頭の 3 人のうち、踊りのうまさ最大の人と最小の人が抜けて、残った 1 人が列の末尾に移動する」操作が、残り 1 人になるまで行われる。
 - 残った 1 人の踊りのうまさとして考えられる最大値を求めよ。
-
- $3 \leq N \leq 99,999$

例 (順番が決まる前)

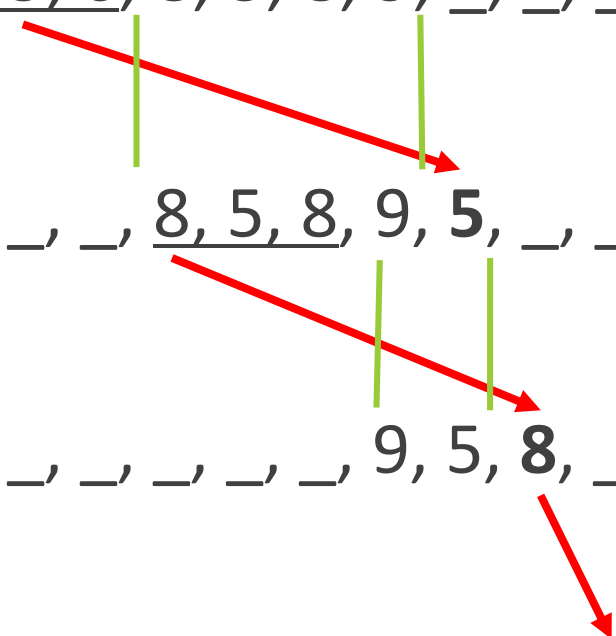
- $[?, 5, ?, ?, 5, 8, ?]$ (6, 2, 5, 9 が入る)

例 (順番が決まった後)

- [2, 5, 6, 8, 5, 8, 9]

- [8, 5, 8, 9, 5]

- [9, 5, 8]

- [8]

処理の変更

- [2, 5, 6, 8, 5, 8, 9, _, _, _]
 - [_, _, _, 8, 5, 8, 9, 5, _, _]
 - [_, _, _, _, _, _, 9, 5, 8, _]
 - [_, _, _, _, _, _, _, _, _, 8]
- 

- 列の更新をする際に、毎回配列全体をスライドさせていると毎回 $O(N)$ かかってしまうが、左図のように最初に大きな配列を用意して次の処理をさせるようにすると列の更新についての計算量が $O(1)$ になります。
- このように処理を変更することによって列が決まった後のシミュレーションのコストが $O(N)$ にできます。

部分点解法 1

- すべての並び順を試します。
- 考えられる並び順は $O(N!)$ 通りあり、これらをシミュレートして、得られた解の最大値を出力する方針で正解を得ることができます。
- 計算量は $O(N!)$ となり、小課題 1 に正解することができます。

方針の転換 1

- どの数字を使って、どの数字を使っていないかを保存すると大変です。
- もしも、登場する数が 2 種類 (high, low) しかなかった場合はどうなるか考えてみます。

high, low しかないとき

- high, low しかない場合でも、以降の項に持っていく値を定めることができます。
 - [high,high,high] → high (high のみなら、次は high)
 - [high,high,low] → high (high 2 つ low 1 つなら、次は high)
 - [high,low,low] → low (high 1 つ low 2 つなら、次は low)
 - [low,low,low] → low (low のみなら、次は low)
- この場合、最初に置く組み合わせは 2^N 通りしかないので、すべての組み合わせを試しても計算量は $O(N * 2^N)$ に抑えることができます。
- この性質をうまく利用できないか考えてみます。

方針の転換 2

- 実際に登場する数は 2 種類とは限らず、もっと多い場合があります。
- もしも、答えが最大値を求めるという最適化問題ではなく、ある整数 K に対して「 K 以上にできますか」という判定問題だった場合を考えてみます。
- こちらの問題の場合、 K 以上である数が何であるかの違い (例 : $K+1$ か $K+2$ かの違い) は考えなくて良くなります (両者を入れ替えても同じ、 K 未満同士についても同様)。
- この場合、 K 以上である数を high、 K 未満である数を low であるとして考えることで、先ほどの探索を行えば $O(N * 2^N)$ で判定することができます。

部分点解法 2

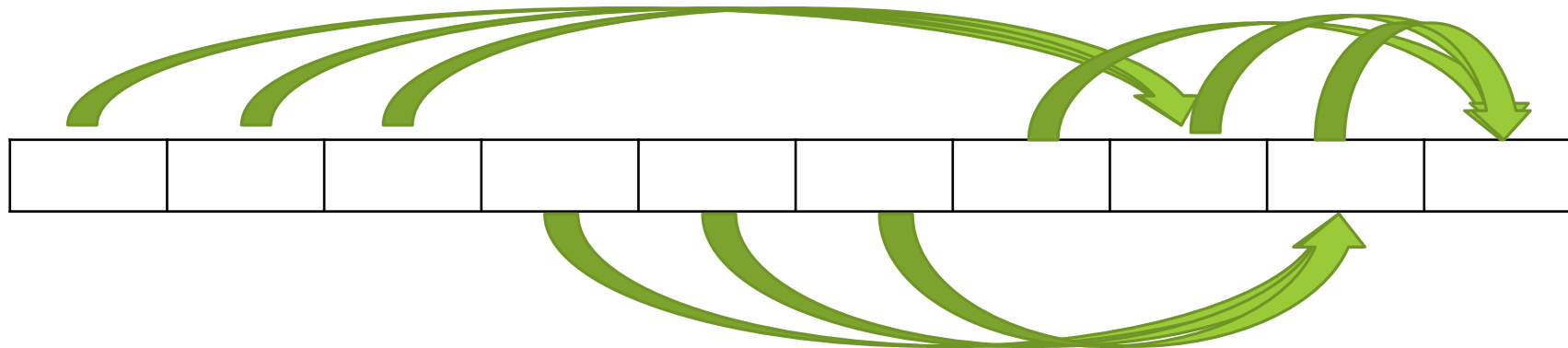
- 答えの候補は高々 N 個しかありません。
- 各候補について、その値を先ほどの K において、high, low の全部の配置を試す方針ならば、全体で $O(N^2 * 2^N)$ で判定できます。
- high を持って行くことのできた K の中で最大のものが答えとなります。

多項式にしたい

- high, low に分ける方針でも、すべての並べ方を試す方針だと、指数時間かかってしまいます。
- 多項式の計算量にするために、操作の性質を調べます。

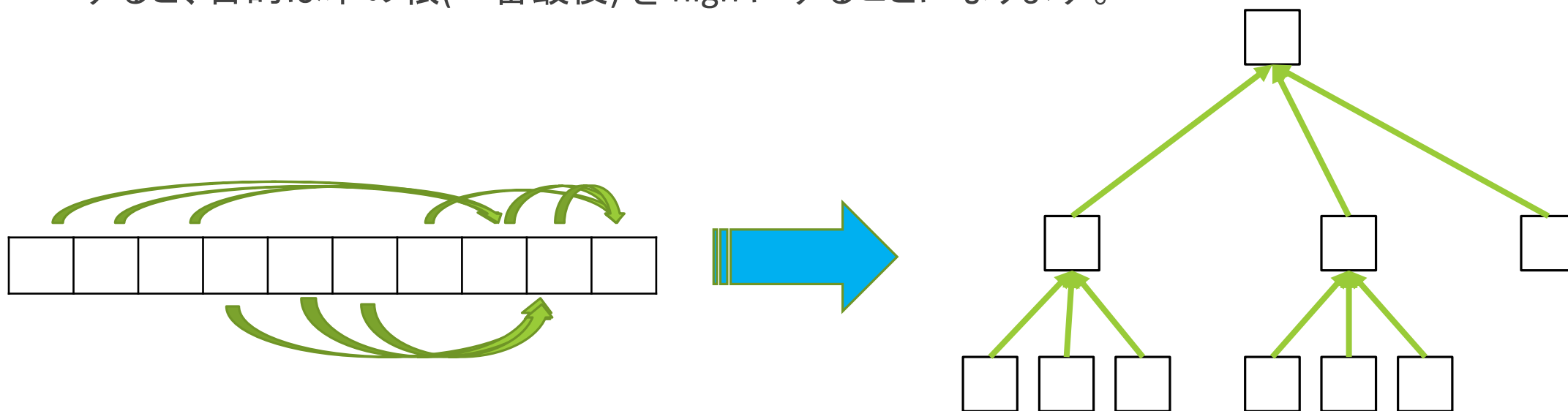
考察

- 配列の各要素について、配列のどの場所が関与するのかは、配列に入る値によらず一定です。



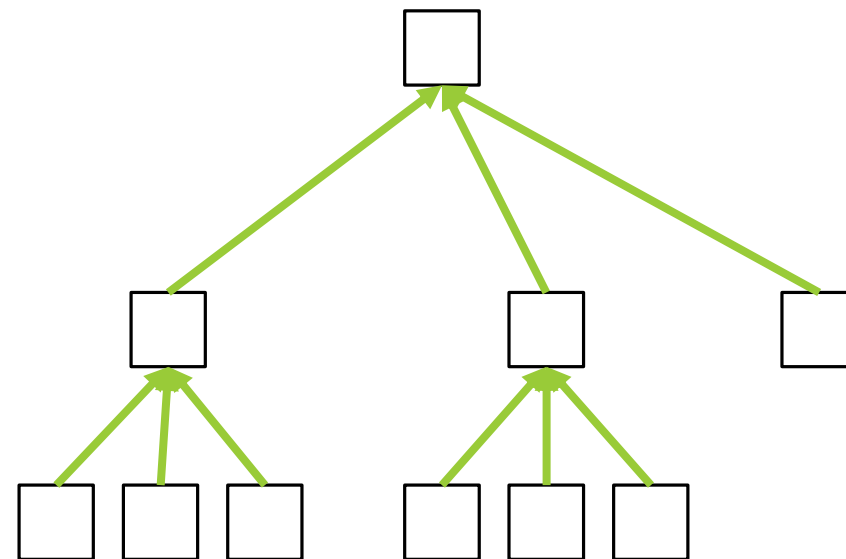
考察

- 先ほどの関係性は、以下のように木構造に変換することができます。
- 以降は、この木構造について話を進めることにします。
- すると、目的は木の根(一番最後)を high にすることになります。



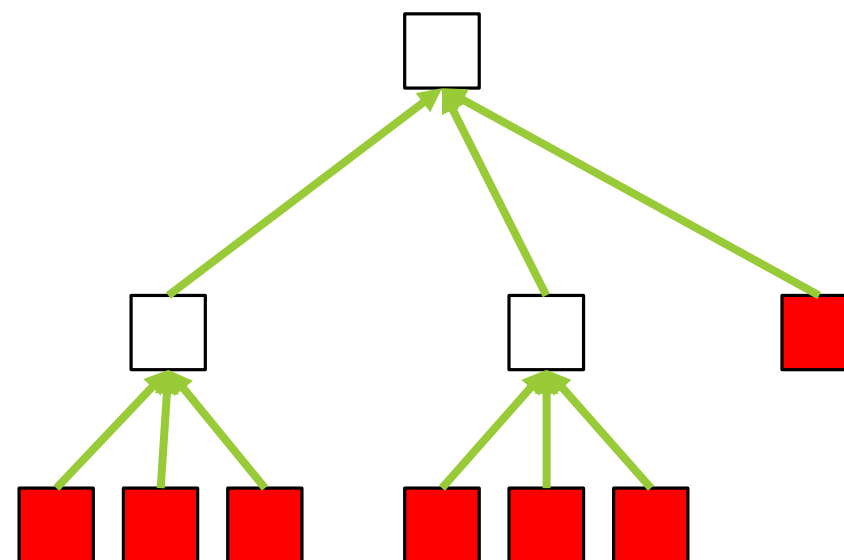
high の節約

- 各ノードについて、そのノードを high にするときに、空いたマスに埋める high の数が少ないほど、他の場所に多く high を回すことができます。
- high は節約した方が良いので、「各ノードについて、そのノードを high にするために必要な high の個数」を計算することを考えます。



葉ノードについて

- 葉ノードについて、そのノードを high にしたい場合は、
埋まっていない → 1 個必要
埋まっていて、high → 0 個必要
埋まっていて、low → INF (high にできない)
となります。



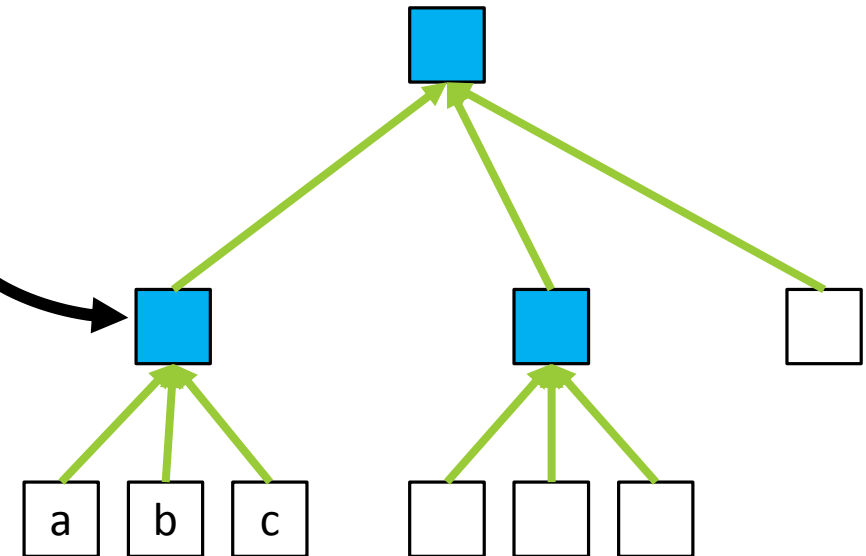
葉ノード以外について

- 葉ノード以外については、そのノードに影響を与える2つのノードのうち、2つが high であれば high になります。一方で high が1個以下なら必ず low になります。
- よって、3つのノードそれぞれについて必要な個数を a 個, b 個, c 個とおくと、

$$a + b + c - \max\{a, b, c\} \text{ (小さい方 2 つ)}$$

となります。

- このようにすることで、動的計画法で、根ノードを high にするのに必要な high の個数の最小値がわかります。



部分点解法 3

- 先ほどの動的計画法は、 $O(N)$ で実行することができます。
- 根ノードの値が、自由における high の個数以下ならば、最初に定めた K 以上にすることができます。
- すべての K について試すことにして、実現可能だった K の中で最大のものを出力することで正解を得ることができます。
- 計算量は $O(N^2)$ となります。

考察

- 例えば、ある K についてダメだったとして、 $K+1$ で実現できるかどうかを考えてみます。
- 初期状態については状況がより悪くなっている (少なくとも良くなっている) はない、手持ちの $high$ もより少なく (あるいは変化なく) なっています。
- このことから、「ある K についてダメだったら、その K より大きい値はいずれもダメ」とわかります。
- 同様に考えてみると、「ある K について成立したら、その K より小さい値はいずれも成立」とわかります。
- 以上 2 つをまとめると、「ある x が存在して、 K が $x \geq K$ なら成立し、 $x < K$ なら成立しない」ということができます。この x が求める答えとなります。
- このような x を求める方法とは...?

二分探索

- 二分探索は、先ほどの X みたいに、「ある値以上か未満かで評価値が変わる関数の境界値を効率的に求める手法」です。
- 具体的には X の範囲が $[a, b]$ (a 以上 b 以下) にあるとわかっている場合に、 $c = (a+b)/2$ (端数切り捨て)として、

$c \geq X$ と分かった $\rightarrow [a, c]$ について再帰的に試す

$c < X$ と分かった $\rightarrow [c+1, b]$ について再帰的に試す

を繰り返して幅を縮める手法です。

※今回、値は全て整数値であると仮定しています。

二分探索の例

- $X=58$ (ここは非公開とします), X が $[1,100]$ に収まっていることがわかっているとします。
- $a=1, b=100$ より $c=50$ で $c < X$ なので区間が $[51,100]$ に縮まります。
- $a=51, b=100$ より $c=75$ で $c \geq X$ なので区間が $[51,75]$ に縮まります。
- $a=51, b=75$ より $c=63$ で $c \geq X$ なので区間が $[51,63]$ に縮まります。
- $a=51, b=63$ より $c=57$ で $c < X$ なので区間が $[58,63]$ に縮まります。
- $a=58, b=63$ より $c=60$ で $c \geq X$ なので区間が $[58,60]$ に縮まります。
- $a=58, b=60$ より $c=59$ で $c \geq X$ なので区間が $[58,59]$ に縮まります。
- $a=58, b=59$ より $c=58$ で $c \geq X$ なので区間が $[58,58]$ に縮まります。
- こうして $X = 58$ だとわかります。

満点解法

- 今回の場合、解の候補は 1 以上 1,000,000,000 以下であることが制約上わかるので、二分探索が有限回で終了し正解を得ることができます。
- 区間の長さが毎回半分ずつに狭まっているので、反復回数が $\log(1,000,000,000)$ 回程度(2を底とする)になります。
- 各 c について x との比較結果を算出するのに、さっきの動的計画法分の計算量 $O(N)$ がかかるので、全体で $O(N \cdot \log(\text{値の範囲}))$ の計算量となります。
- $\log(\text{値の範囲})$ という値は 30 くらいと小さく、満点を得ることができます。
- 範囲が大きくても、解の候補が高々 N 個しかないので、 $O(N \log N)$ にできます。

おまけ

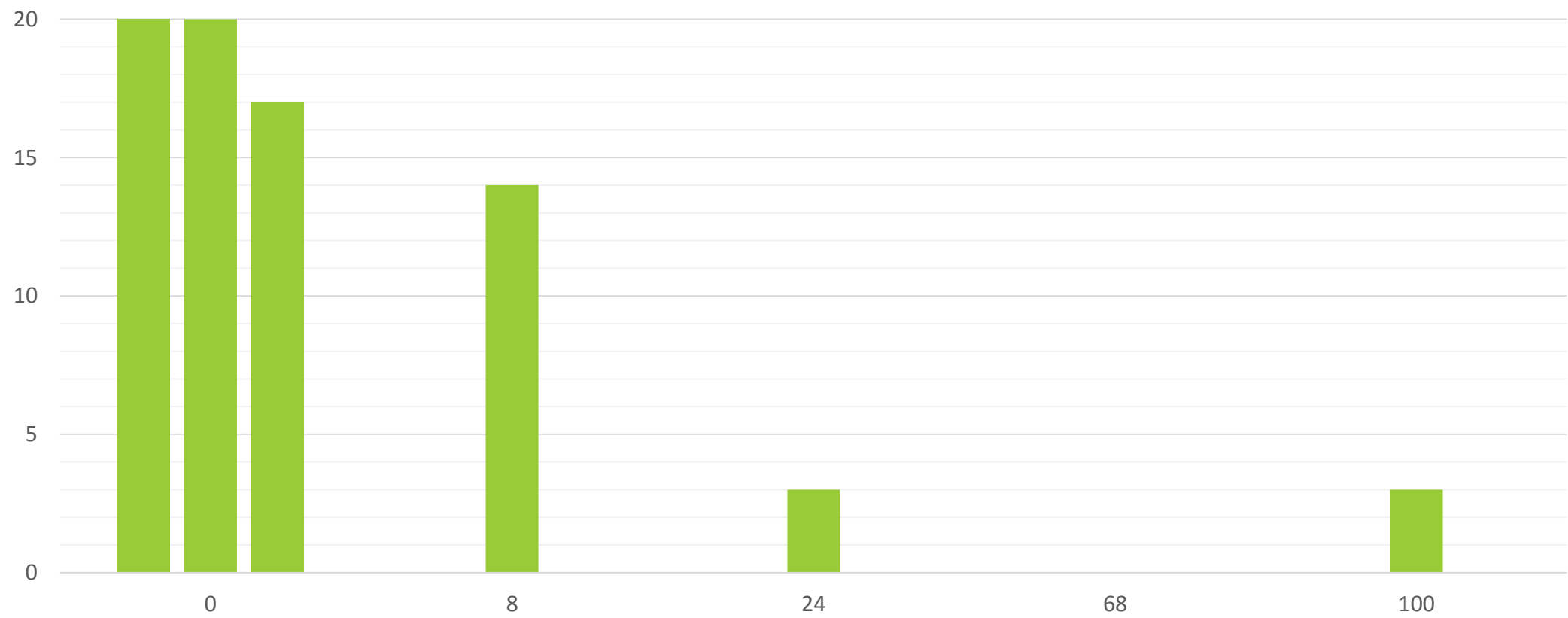
乱択アルゴリズム

- 「 $N!$ の並び替えのうち無作為に 1 つ選んでシミュレーション」を時間のぎりぎりまで繰り返して最大スコアを出力する。
- この手法だと一体どのくらいの得点が望めるのでしょうか？

実はそこそこ強い

- 適当に並べ替える操作は、「答えとなる X に対しての high, low の割り当て方のうち無作為に並べ替えたものの 1 つを実験する操作」と等しいので、1 回あたり $1/2^N$ の確率で正解を得ることができると見積もることができます。
- 実際はそれよりもはるかに少なく、 $1/N \cdot C(\frac{N}{2})$ 以上の確率で正しい割り当てをしていることになります。
- subtask 2 までの範囲ならば有効です。
- このように乱択アルゴリズムが時として有用な場合があるので、いろいろ試してみると、思わぬ発見があるかもしれません。

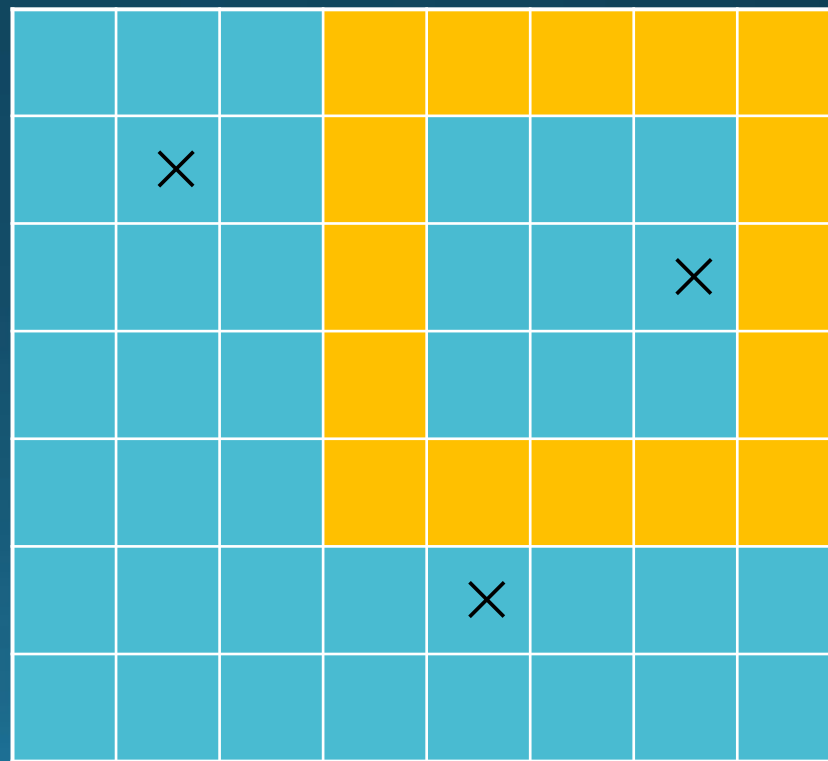
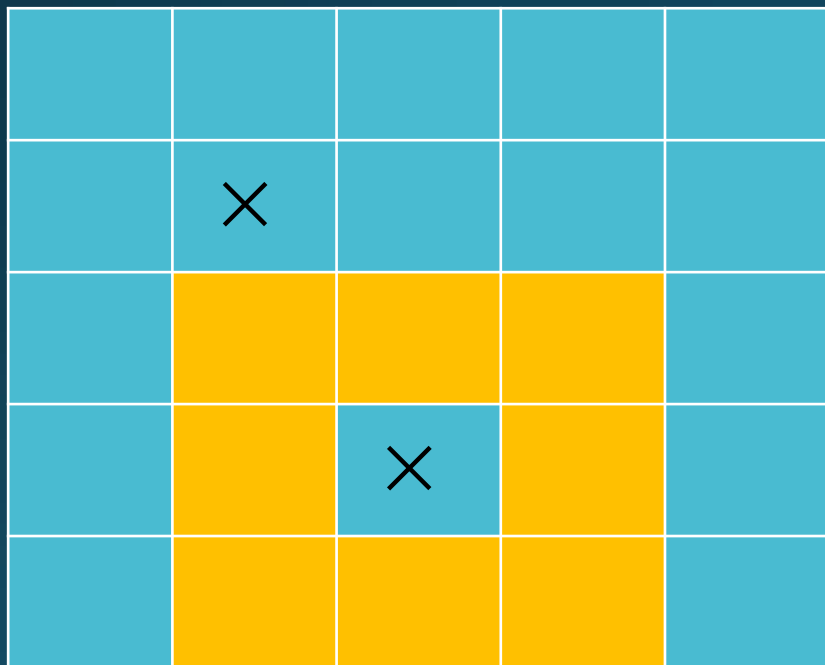
得点分布



本選 問 5 Rampart

問題概要

- グリッドが与えられます
- 幅₁の（十分大きい）正方形の枠の置き方は何通り？



小課題 1

- 置けない位置を無視すると，調べる位置は $O(HW \min(H, W))$ 個
- それぞれの位置について， $O(1)$ でチェックできればよい
- 累積和を使う

累積和

- 詳しいやり方は参考書（蟻本など）を見てください
- 累積和を適切に使うと，前処理 $O(HW)$ で，2次元配列のある長方形領域の値の和が $O(1)$ で求められる
- 2次元配列として，「その位置に城壁がなかったら 1, あったかもしれないなら 0」とした配列をとる
- 枠を決めたとき，枠の上の「そこには城壁がなかった」マス
の数は $O(1)$ で求められる

入出力例 1

	×			
		×		

↑ 置けないマス

0	0	0	0	0
0	1	0	0	0
0	0	0	0	0
0	0	1	0	0
0	0	0	0	0

↑ 配列

入出力例 1

0	0	0	0	0
0	1	1	1	1
0	1	1	1	1
0	1	2	2	2
0	1	2	2	2

↑ 累積和

0	0	0	0	0
0	1	0	0	0
0	0	0	0	0
0	0	1	0	0
0	0	0	0	0

↑ 配列

入出力例 1

- の橙色の部分に 1 が何個あるか知りたいとき

0	0	0	0	0
0	1	0	0	0
0	0	0	0	0
0	0	1	0	0
0	0	0	0	0

入出力例 1

- ↓ の橙の部分の和から

0	0	0	0	0
0	1	0	0	0
0	0	0	0	0
0	0	1	0	0
0	0	0	0	0

- ↓ の赤の部分の和を引く

0	0	0	0	0
0	1	0	0	0
0	0	0	0	0
0	0	1	0	0
0	0	0	0	0

小課題 1

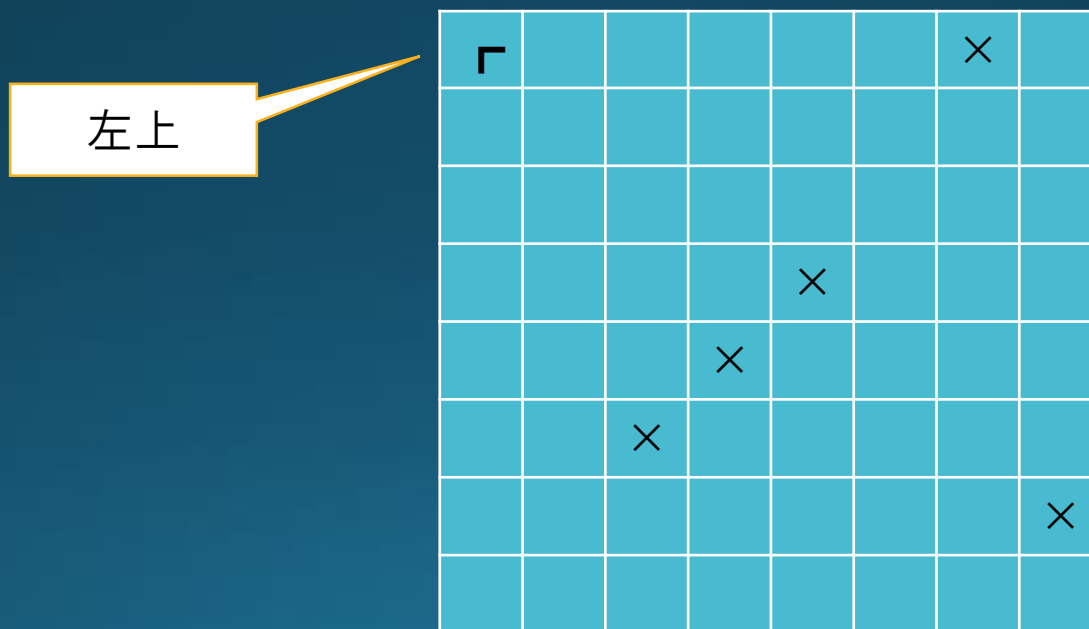
- こうやると， 枠を決めた時のチェックが $O(1)$ でできる
 - 前処理 $O(HW)$
 - 調べる枠候補の数は $O(HW \min(H, W))$
 - 合計 $O(HW \min(H, W))$
-
- 小課題 1 が通って 4 点 that 得られる

小課題 2

- $P \leq 10$
- 「置けないマス」の数が異様に少ない
- ところで、置けないマスがまったく存在しなければ答えは簡単に求められる
- この小課題では、「だめな枠の数」を数えるとよさそう

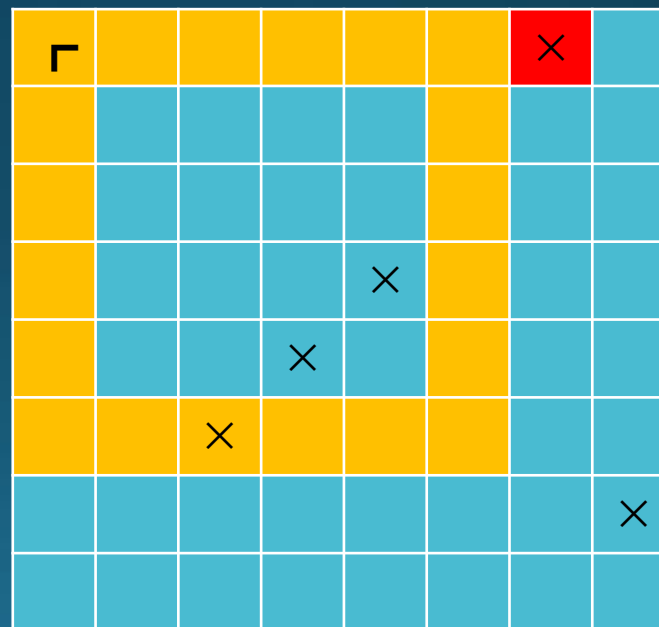
小課題 2

- 枠の左上を (a, b) に固定したときに，だめな枠が何個あるか数える



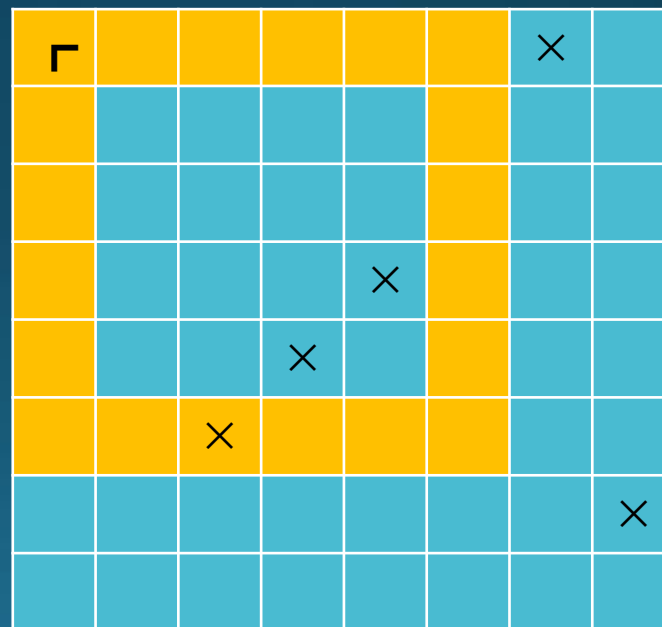
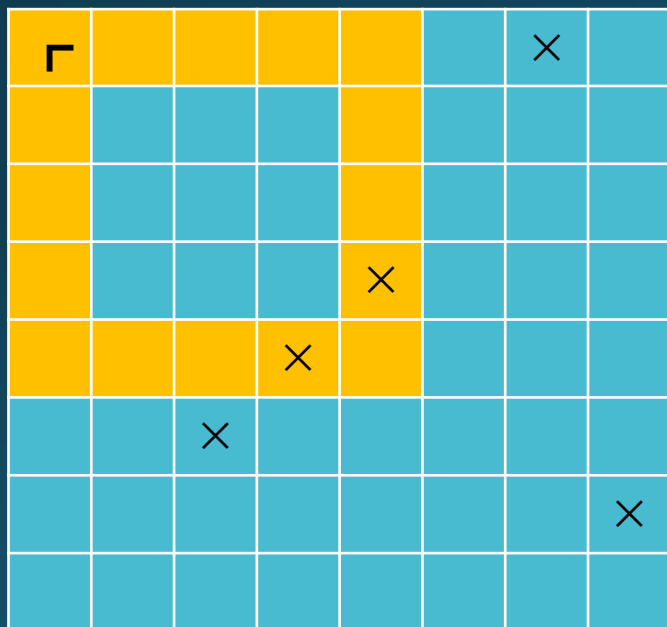
小課題 2

- 「左上」から，下や右にたどって行って，だめなマスにぶつかるところがあったらそこが枠の大きさの限界



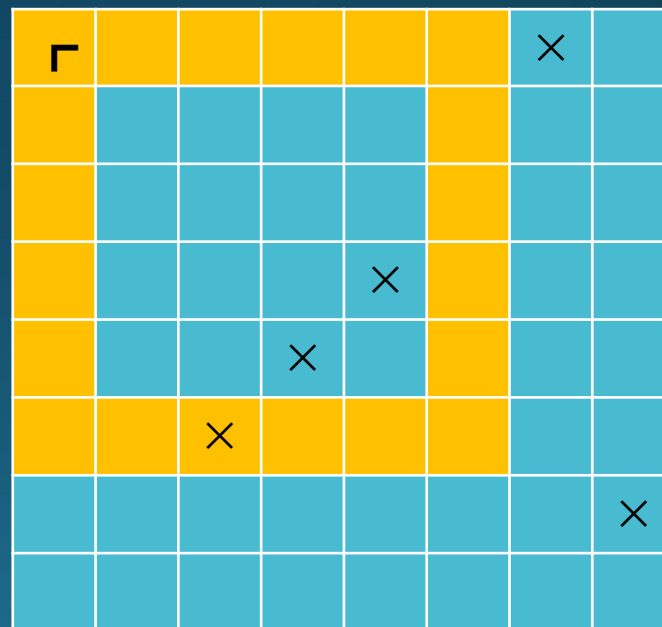
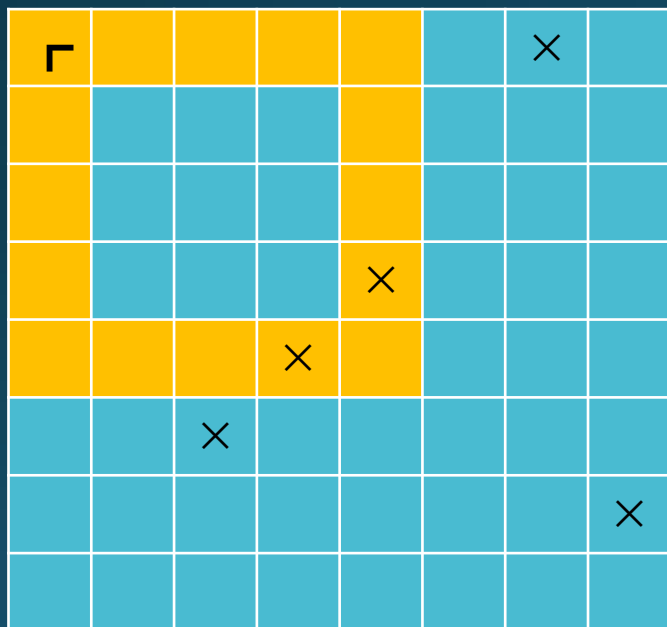
小課題 2

- それ以外で，右下にある「だめなマス」は，枠の大きさを動かすとちょうど1回だけぶつかる



小課題 2

- ぶつかる大きさを列挙して，下限 (L) と上限 (左上から伸ばした辺が，壁かだめなマスにぶつかるまで) の間にあるような大きさが何種類あるか数える



小課題 2

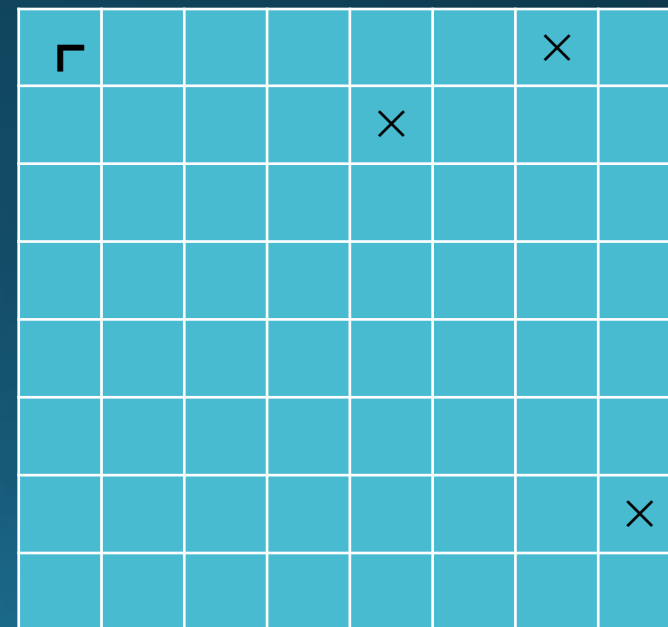
- 同じ大きさでぶつかるようなマスが複数あるかもしれないので注意
- ソートなどによって重複を除く
- 左上の選び方 $O(HW)$
- だめな大きさを調べるのに $O(P \log P)$ (ソートの場合)
- $O(HW P \log P)$ で少しこわい ($>_<$) が余裕で通る
- 16 点が得られる
- P の大きさに場合分けして小課題 1 も解くと 20 点

満点解法の前に

- $4000 * 4000$ だと，答えは最大で 200 億以上になる
- TLE 10 秒とはいえ，答えをいちいち数え上げる余裕はない
- 複数の答えをまとめて数え上げる必要がある

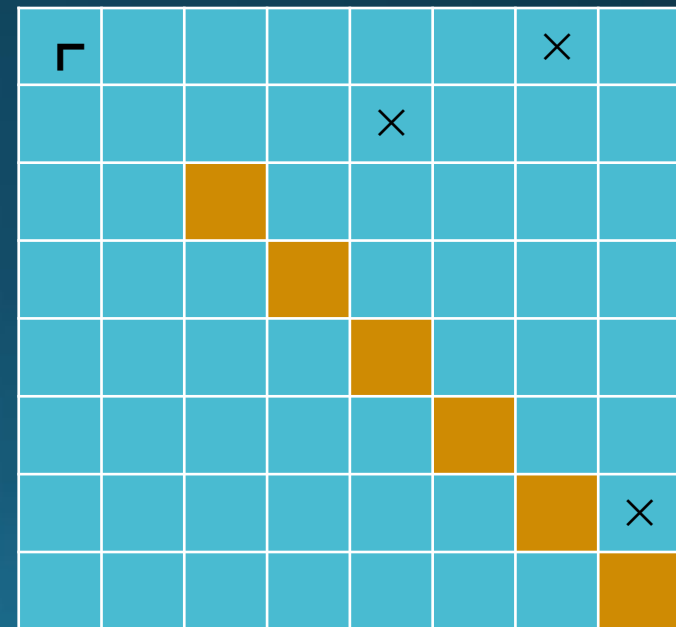
左上を固定

- 変な関係のものたちをいっぺんに数えるのは無理そう
- 左上を固定して，右下を動かして考える



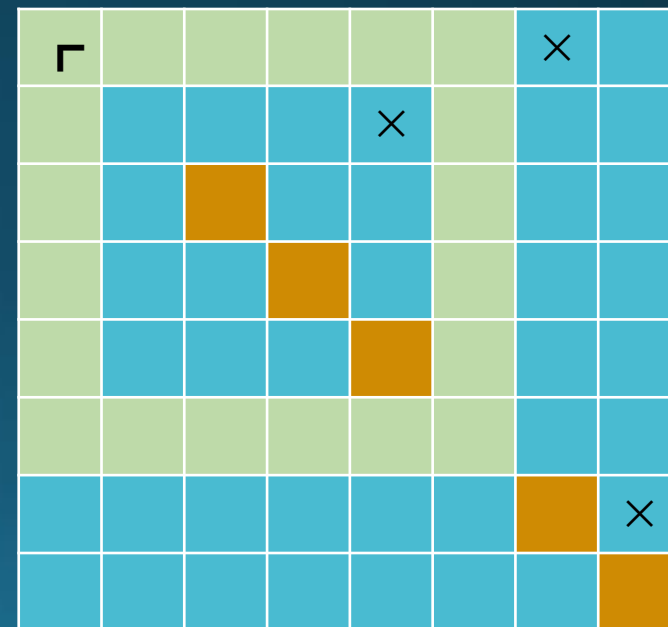
右下の動ける範囲

- 右下は，左上からななめ 45 度の線の上にある
 - 正方形なので当然
- L の大きさによって，あまり近くてはいけない



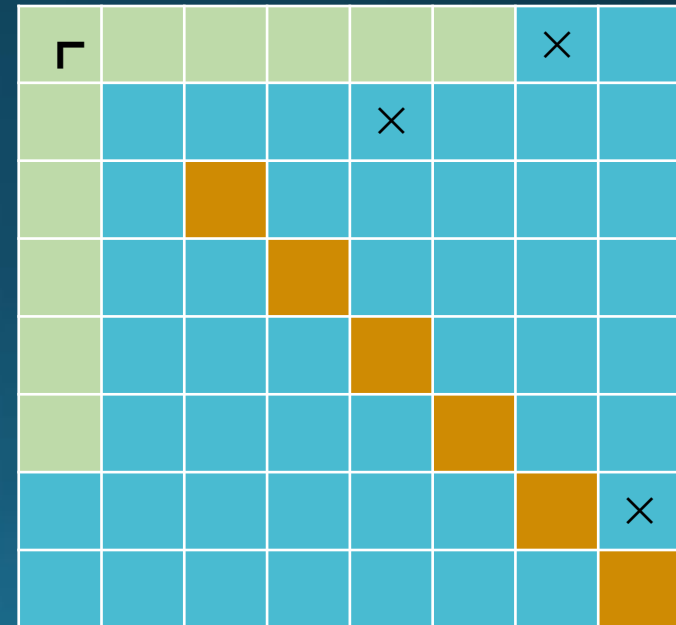
城壁を分けて考える？

- 城壁は枠型
- 左上から伸びる ㄱ型の部分と,
- 右下から伸びる ㄴ型の部分とに分けて考えられる



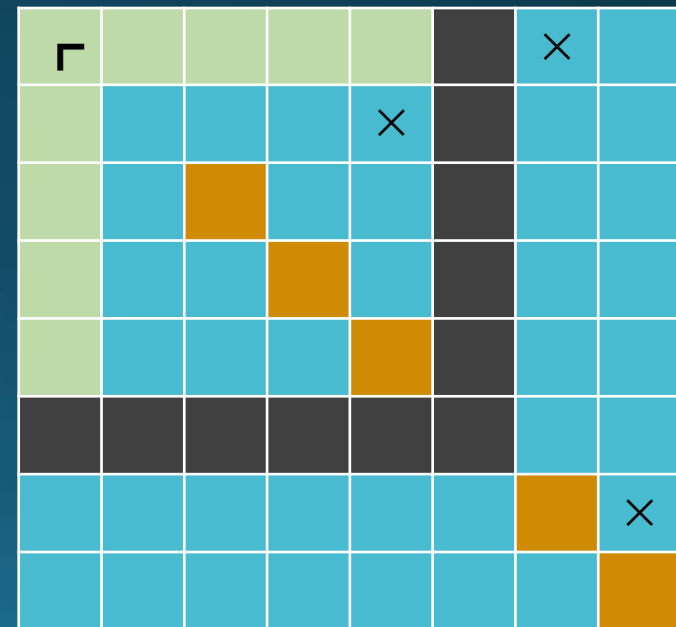
左上部分についての条件

- あまり大きい城壁を作ると，左上から伸びる城壁部分が「外周」「だめなマス」にぶつかってしまう
- 下，右に伸ばしていったって，「外周」「だめなマス」にぶつかるまでだけ考えればよい



右下部分についての条件

- こういうのはOK
- 無事に左上から伸びる部分とぶつかって
枠ができてる

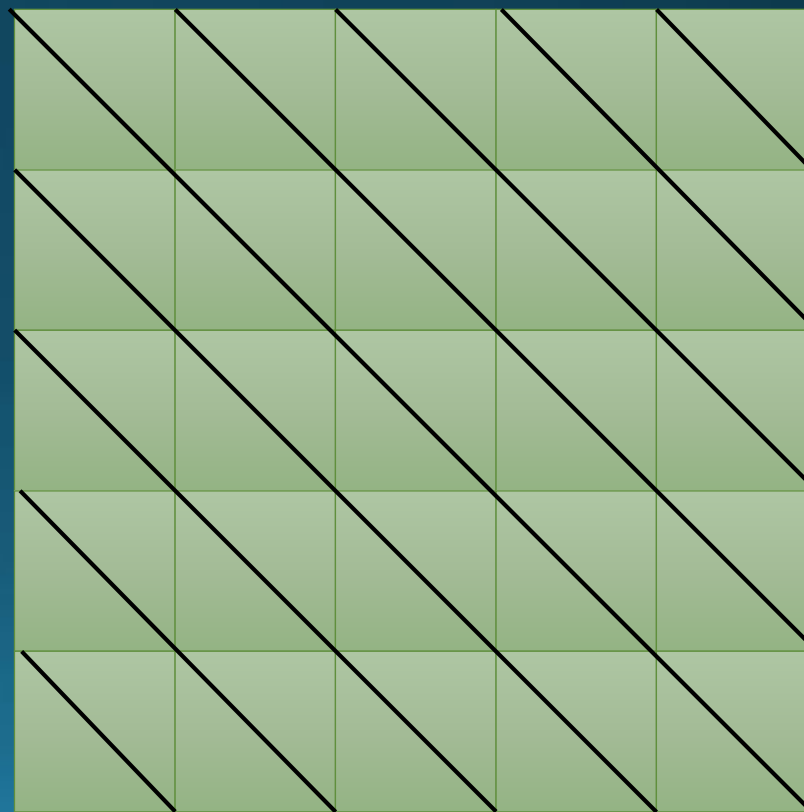


条件まとめ

- 左上/右下から伸ばせる長さの最大値を「限界長さ」と呼ぶことにすると...
- 左上, 右下の間が(左上, 右下含めて)Dマス離れているなら
 - $L \leq D$
 - $D \leq$ 左上の限界長さ
 - $D \leq$ 右下の限界長さ
 - 当然, 左上と右下が同じ斜め線の上にある
- が必要十分

左上，右下の関係

- 同じ斜め線の上にはないといけない
- 斜め線ごとに考えてよさそう



限界長さの計算

- 左上の場合について
- 右に何マス伸ばせるかは DP でわかる

	×			
		×		

				1
				1
				1
				1
				1

限界長さの計算

- 左上の場合について
- 右に何マス伸ばせるかは DP でわかる

	×			
		×		

			2	1
			2	1
			2	1
			2	1
			2	1

限界長さの計算

- 左上の場合について
- 右に何マス伸ばせるかは DP でわかる

	×			
		×		

		3	2	1
		3	2	1
		3	2	1
		0	2	1
		3	2	1

限界長さの計算

- 左上の場合について
- 右に何マス伸ばせるかは DP でわかる

	×			
		×		

	4	3	2	1
	0	3	2	1
	4	3	2	1
	1	0	2	1
	4	3	2	1

限界長さの計算

- 左上の場合について
- 右に何マス伸ばせるかは DP でわかる

	×			
		×		

5	4	3	2	1
1	0	3	2	1
5	4	3	2	1
2	1	0	2	1
5	4	3	2	1

限界長さの計算

- 左上の場合について
 - 下に何マス伸ばせるかも DP でわかる
 - 右に伸ばすのとまったく同様！
 - min をとれば，限界長さが計算できる
 - 右下の場合も，伸ばす方向を上と左にしてまったく同様にやればよい
-
- これで $O(HW)$ で限界長さがすべて計算できる

条件（再掲）

- 左上，右下の間が(左上，右下含めて) D マス離れているなら
 - $L \leq D$
 - $D \leq$ 左上の限界長さ
 - $D \leq$ 右下の限界長さ
- これを満たすような，左上，右下のペアを数えたい

条件（再掲）

- 左上, 右下の間が(左上, 右下含めて) D マス離れているなら
 - $L \leq D \leq$ 左上の限界長さ
 - $D \leq$ 右下の限界長さ
- 上の条件だけだったらなんとかなりそう？
 - 左上を決めた時, L 以上 (限界長さ) 以下の範囲に右下が何個あるか数えらるとよい
 - データ構造でなんとかできそう
- 下の条件がうっとうしい

条件（書き直し）

- 左上, 右下の間が(左上, 右下含めて)Dマス離れているなら
 - $L \leq D \leq \min(\text{左上の限界長さ}, \text{右下の限界長さ})$
- つまり, 限界長さが短い方により D は制限される

条件（もっと書き直し）

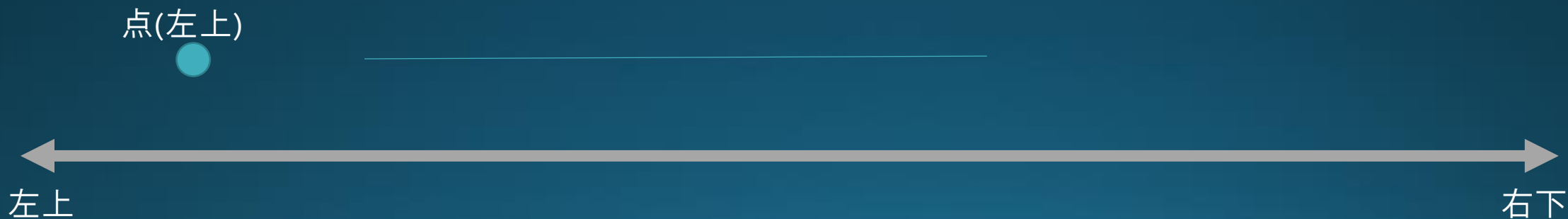
- 左上 / 右下を固定したとき,
- 「 $L \leq D \leq (\text{その点の限界長さ})$ 」をみたす相手に,
!!! 自分より限界長さが長い点 !!!
- を求めたい
- 限界長さが同じときは, 適当に順序を定める

満点解法

- 斜め線を固定して考える
- 線の上の各点を左上 / 右下としたときの限界長さが長い方から順番に見ていく

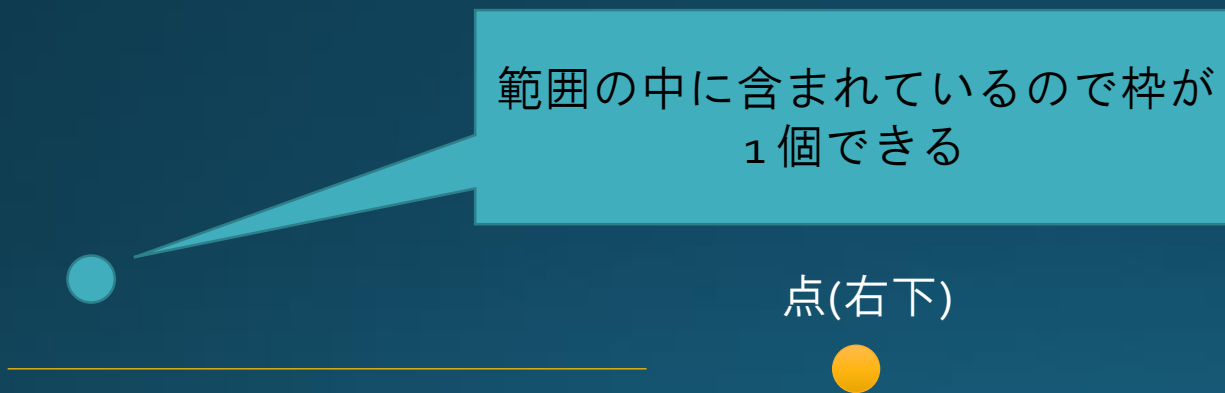
満点解法

- 斜め線を固定して考える
- 線の上の各点を左上 / 右下としたときの限界長さが長い方から順番に見ていく
- 妥当な範囲の中に，ペアとなりうる点は何個あるか数える



満点解法

- 斜め線を固定して考える
- 線の上の各点を左上 / 右下としたときの限界長さが長い方から順番に見ていく



満点解法

- 斜め線を固定して考える
- 線の上の各点を左上 / 右下としたときの限界長さが長い方から順番に見ていく



満点解法

- 斜め線を固定して考える
- 線の上の各点を左上 / 右下としたときの限界長さが長い方から順番に見ていく

左上を固定しているので、
他の左上の点は気にしない



満点解法

- 斜め線を固定して考える
- 線の上の各点を左上 / 右下としたときの限界長さが長い方から順番に見ていく

L の値のため、あまり点に近いものは考えない



満点解法

- 今まで見た (左上 / 右下) の点たちの中に, ある範囲に含まれるものが何個あるか? を数えたい
- BIT (Binary Indexed Tree) が使える
 - 詳細は省略します
 - 蟻本などを見てください
- 点が現れたときには, BIT のその位置のところに 1 を加える
- 点を数えるときは, BIT の範囲クエリを使う

計算量

- H, W が N くらいであるとする
- 限界長さの計算は, $O(N^2)$
- 斜め線は $O(N)$ 個
 - 各斜め線の上に, $O(N)$ 個のマス
 - 限界長さでソートするのに $O(N \log N)$
 - 各マスを見るときに $O(\log N)$ (BIT の操作)
 - 結局, 斜め線ごとに $O(N \log N)$
- あわせて $O(N^2 \log N)$
- 満点が得られる

注意

- $4000 * 4000$ をいっぺんに処理しようとするとは多分時間がかかります
 - $4000 * 4000 * \text{sizeof(int)} = 64\text{MB}$ くらい
 - キャッシュに載らない
- 限界長さの計算をした後は、斜め線ごとに処理を完全に別々に行うのがよい

得点分布

